



목차

테크톤

목차

1. 소개/과정 안내
2. 강사 소개
3. 테크톤 랩 구성 개요
 - Hyper-V 기반 랩
 - OpenStack 기반 랩
4. 에디터/검증 도구/확인 방법
5. Tekton 대시보드



목차

7. 클라우드 네이티브 CI/CD 개요 (Jenkins vs Tekton)

8. Tekton 기본 자원

- Step
- Task / Pipeline
- 기본 자원 구성 패턴
- 기본 자원 소개(공유 디렉터리/Result 등)

9. 디버깅(기본)



목차

7. Workspace로 데이터 공유

- 저장소 형식
- 첫 Workspace 생성
- Workspace에서 작업 실행
- Pipeline에 Workspace 추가
- Pipeline 영구 데이터 저장소 구성



목차

- 11. Pipeline 심화/전체 흐름
- 12. 디버깅(파이프라인/태스크 정리)
- 13. 조건식(When 등)
- 14. 인증(기본 인증/SSH)
- 15. 기능 확장(Hub)
- 16. Trigger
- 17. 최종 랩



소개

과정

담당자 및 강사

소개 및 요구사항

01

다음과 같은 조건의
교육을 진행 합니다.

- CNCF Native 기반
- 최신 버전 쿠버네티스 기반
- tkn 명령어 및 설치

02

기본적인 쿠버네티스
경험이 있는
사용자에게 권장

03

젠킨스에서 CNCF 표
준으로 마이그레이션을
원하는 시스템/소프트
웨어 엔지니어



강사

강사

강사

이름: 최국현

메일: tang@dustbox.kr, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

언제든지 질문 및 요청 환영 입니다.



LAB

랩 및 교육대상

테크톤 교육 대상

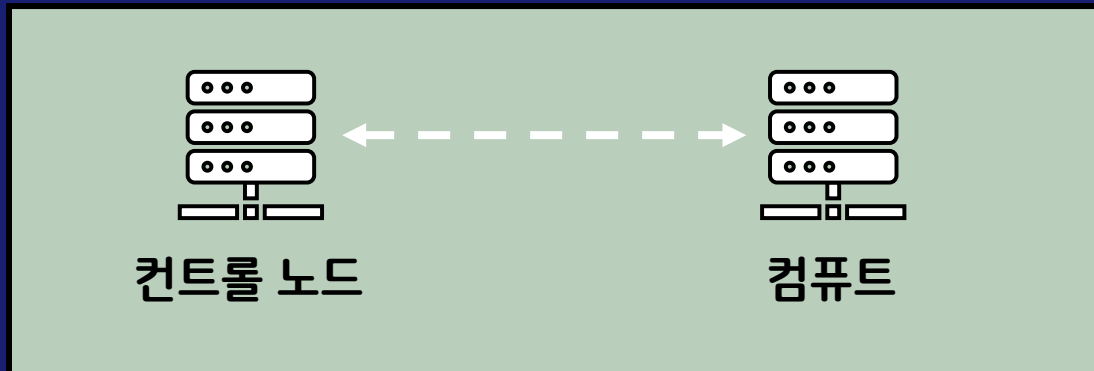
이번 앤서블 교육은 다음 대상으로 제작 및 구성.

1. 앤서블 처음 사용하는 사용자
2. 실무까지는 아니어도 어떠한 방식으로 동작하는지 궁금하신 사용자
3. 간단하게 YAML 형태로 동작 학습
4. 전체적인 용어 및 기능 학습



가상머신 구성

자원 상태에 따라서 다르지만 한 대 혹은 두 대 서버로 진행이 가능하다. 추가적으로 유틸리티 서버가 필요하다.
이 부분은 강사의 가이드에 따라서 구성한다.



```
control/compute nodes  
ens3: DHCP
```

```
control/compute nodes  
control[ens4]: 10.10.20.1  
compute[ens4]: 10.10.20.2
```

테크톤 HelmChart설치

HelmChart를 통해서 Tekton설치를 진행한다. 기본적인 도구는 스크립트로 설치가 진행이 되며, 트리거와 같은 부분은 별도로 설치해야 한다.

```
helm repo add tekton https://cd.foundation.github.io/tekton-helm-chart/  
helm repo update  
helm install tekton-pipelines tekton/tekton-pipeline --namespace tekton-  
pipelines --create-namespace
```

```
kubectl apply --filename https://storage.googleapis.com/tekton-  
releases/triggers/latest/release.yaml  
kubectl apply --filename https://storage.googleapis.com/tekton-  
releases/triggers/latest/interceptors.yaml
```



테크톤 HelmChart설치

테크톤 대시보드가 필요한 경우 아래처럼 추가로 설치한다.

```
kubectl apply -f  
https://github.com/tektoncd/dashboard/releases/latest/download/tekton-  
dashboard-release.yaml
```

```
kubectl get crds | grep tekton  
kubectl get pods -n tekton-pipelines
```



에디터 및 확인

테크톤 랩 구성

VI/VIM

에디터

간단한 에디터 설정(vim)

간단한 방법은 아래 패키지만 설치한다. 레드햇 계열은 9버전 이후부터 해당 패키지가 존재한다.

```
dnf install -y vim-ansible
```



복잡한 에디터 설정(vim)

파일 작성을 원활하게 하기 위해서 `yamllint`, `ale` 그리고 `neovim` 기반으로 진행한다. `neovim`과 `vim+ale`은 동작 방식이 다르기 때문에 선호하는 방식으로 선택 후 설치를 진행한다.

```
curl -sS https://webi.sh/vim-ale | sh
dnf install epel-release -y
dnf search yamllint
yamllint.noarch : A linter for YAML files
```

```
dnf install yamllint -y
dnf install neovim-ale -y
```



.vimrc

VIM, NeoVIM은 둘 다 같은 설정 파일을 공유한다. VI계열을 사용하는 경우 아래와 같이 옵션을 설정한다. 다만, ALE, YAMLLINT는 꼭 설치가 되어 있어야 한다.

```
cat <<EOF> ~/.vimrc
autocmd FileType yaml setlocal ts=2 sts=2 sw=2 expandtab

set foldlevelstart=20

let g:ale_echo_msg_format = '[%linter%] %s [%severity%]'
let g:ale_sign_error = 'X'
let g:ale_sign_warning = '⚠'
let g:ale_lint_on_text_changed = 'never'
EOF
```



NANO

에디터

에디터 설정(nano)

나노 에디터 사용하는 경우 아래처럼 작업을 수행한다.

```
dnf search nano
dnf install nano -y
cat <<EOF> ~/.nanorc
set tabsize 2
set tabstopspaces
EOF
```



에디터 설정(nano)

나노 에디터 사용하는 경우 아래처럼 작업을 수행한다.

```
cat <<EOF> /usr/share/nano/yaml.nanorc
syntax "YAML" "\.ya?ml$"
header "^(---|==)" "%YAML"
color magenta "^\\s*[\\$A-Za-z0-9_-]+\\: "
color brightmagenta "^\\s*@[\\$A-Za-z0-9_-]+\\: "
color white "\\:\\s+ $"
color brightcyan " (y|yes|n|no|true|false|on|off) $"
color brightred " [[[:digit:]]+(\\. [[[:digit:]]+)? "
color red "\\[" "\\]" "\\:\\s+[>]" "^\\s*- "
color green "(^| )!!(binary|bool|float|int|map|null|omap|seq|set|str) "
color brightwhite "#.* $"
```

계
속



에디터 설정(nano)

나노 에디터 사용하는 경우 아래처럼 작업을 수행한다.

```
color ,red ":\w.+$"  
color ,red ":' .+$"  
color ,red ":' .+$"  
color ,red "\s+$"  
color ,red "['\" ]^[ '\" ]*$"  
color yellow "['\" ].*['\" ]"  
color brightgreen ":( |$)"  
EOF
```



tkn CLI

테크톤 명령어는 다음과 같이 설치 및 자동완성 활성화가 가능하다.

```
tkn completion bash > /etc/profile.d/tkn.bash  
source tkn.bash
```



TMUX

여러 콘솔이 필요한 경우, 아래와 같이 tmux를 통해서 사용 및 적용이 가능하다.

```
dnf install tmux -y
cat <<EOF> ~/.tmux.conf
set -g mouse on
set -g prefix C-a
unbind C-b
bind C-a send-prefix
EOF
```



에디터 테스트 hello.yaml

에디터 및 테크톤이 올바르게 동작하는 확인하기 위해서 **hello.yaml**파일을 생성한다.

```
cat <<'EOF' | kubectl apply -f hello.yaml
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: hello
spec:
  steps:
    - image: quay.io/centos/centos
```



hello.yaml

적용 및 올바르게 동작하는지 확인한다. `tkn` 명령어가 없는 경우 다운로드 받아서 적절한 위치에 설치한다.

```
command:
  - /bin/bash
  - -c
  - echo "Hello World"
```

EOF

```
kubectl apply -f hello.yaml
```

```
mkdir ~/bin/
```

```
tar xzf tkn_0.32.0_Linux_x86_64.tar.gz -C ~/bin/
```

```
tkn task list
```

NAME	DESCRIPTION	AGE
hello		55 seconds ago



테크톤 대시보드

DASHBOARD

Tekton dashboard

대시보드 접근은 아래와 같은 방법으로 가능하다.

```
kubectl --namespace tekton-pipelines port-forward svc/tekton-dashboard  
9097:9097
```

```
kubectl --namespace tekton-pipelines port-forward svc/tekton-dashboard --  
address <ETH0_IP_ADDRESS> 9097:9097
```

```
host]# firefox http://<ETH0_IP_ADDRESS>:9097/
```



Tekton dashboard

The screenshot displays the Tekton Dashboard interface. On the left is a sidebar with navigation links: Tekton resources, Pipelines, PipelineRuns (selected), PipelineResources, Tasks, ClusterTasks, TaskRuns, Conditions, EventListeners, TriggerBindings, ClusterTriggerBindings, TriggerTemplates, and About. The main content area shows a specific PipelineRun titled 'dashboard-release-nightly-drt9f' from 2 days ago. It indicates a 'Succeeded' status with 2 tasks completed and 0 failed, cancelled, or skipped. A list of tasks is shown on the left, including 'build', 'publish-images', and various sub-tasks like 'create-dir-builtashb...', 'create-dir-dashboa...', 'source-copy-dashboa...', 'create-dir-bucket-for...', 'fetch-bucket-for-das...', 'link-input-bucket-to...', 'ensure-release-dirs...', 'dashboard-run-ko', 'copy-to-latest-bucket', 'tag-images', 'image-digest-exporte...', 'source-mkdir-bucket...', 'source-copy-bucket-f...', and 'upload-bucket-for-da...'. The 'dashboard-run-ko' task is highlighted, showing a 'Completed' status with a duration of 7 minutes 31 seconds. Below this, there are tabs for 'Logs', 'Status', and 'Details'. The 'Logs' tab is active, displaying a terminal output that includes commands for activating service account credentials, configuring Docker, and running 'ko' to build and push images. The output also shows a warning about credential helpers and a notice to install 'ko' from GitHub.



테크톤 소개

CNCF TEKTON

TEKTON

테크톤은 CNCF에서 공식으로 지정한 Continuous Delivery 도구이다.

테크톤은 엄밀히 말하자면 절차적인 자동화 도구라고 보시면 된다. 테크톤은 Continuous Integration 기능도 제공하나, SCM에 구성이 되어 있는 소스코드를 가져와서 이미지 빌드를 할 수 있도록 지원한다.



TEKTON

테크톤은 개발자에게 다음과 같은 기능 제공한다.

1. build
2. test
3. deploy

테크톤에서 모든 자원은 쿠버네티스와 동일하게 **YAML**기반으로 제공한다. 코드 기반으로 각각 작업들을 구성하며, 해당작업을 작업공간 및 파이프라인을 통해서 처리가 가능하다.

개발자가 손쉽게 YAML코드 기반으로 애플리케이션을 서비스로 배포가 가능하다.



TEKTON

테크톤은 CNCF사이트 소개가 되어 있다. **NONE-PROFIT 오픈소스 프로젝트**이다. 테크톤은 CNCF에서 아직 GRADUATED는 아니지만, CNCF에서 오픈소스로 보증하는 프로젝트이다.

ArgoCD는 CNCF에서 **GRADUATED**된 프로젝트이다.

The image shows two overlapping screenshots from the Cloud Native Computing Foundation (CNCF) website. The background screenshot is the 'Organization' page for the Continuous Delivery Foundation (CDF), which is a NOT PROFIT organization based in San Francisco, California, United States. It describes itself as 'A Neutral Home for the Next Generation of Continuous Delivery Collaboration' and lists statistics: Funding is '-', Employees are '1 - 10', and Storage is 'Storage'. The foreground screenshot is the project page for Argo, a Cloud Native Computing Foundation (CNCF) project. It is a TAG APP DELIVERY project, established in 2020, and is currently in the GRADUATED state. The page describes Argo as 'Kubernetes-native tools to run workflows, manage clusters, and do GitOps right.' It includes a maturity timeline showing the progression from SANDBOX to GRADUATED, with key dates: 2020-03-26 for INCUBATING and 2022-12-06 for GRADUATED. The 'Repositories' section lists 'argoproj/argo-cd (primary)' with a link to 'https://github.com/argoproj/argo-cd'. It also shows the project's license as 'Apache License 2.0', its status as 'PRIMARY', and mentions 'good first issues' and '29 open' issues.



TEKTON VS JENKINS

테크톤과 젠킨스는 기능 및 역할이 다르다. **젠킨스는 플러그인 및 혹은 로컬 언어(Local Language)**를 사용해서 작업 순서를 구성한다.

젠킨스 Groovy라는 스크립트 명령어를 통해서 파이프라인 및 작업 수행이 가능하지만, 간단하게 파이프라인 및 자바 JDK와 같은 도구를 지원한다. 다른 의미로 테크톤 보다 더 제한적인 범위로 지원한다.

처음 시작하는 사용자에게는 학습 난이도가 낮은 테크톤 기반으로 구성을 권장하며, 추가적인 기능이 필요한 경우 플러그인 제작 및 설치가 별도로 필요하지 않는다. 또한, YAML문법을 사용하기 때문에 젠킨스의 Groovy언어 문법 및 구조체를 복잡하게 학습할 필요가 없다.



TEKTON VS JENKINS

젠킨스 Groovy 문법 형식은 아래와 같다. 하양식이 아닌, 선언 및 문법을 통해서 Groovy코드를 작성해야 한다. 물론, 선언 형식에 대해서도 학습이 필요하다. 그와 반대로 테크톤은 YAML형식으로 단순하게 선언 및 작성이 가능하다.

```
stages {
  stage('Checkout') {
    steps {
      git url: "${GIT_URL}", branch: "${GIT_BRANCH}"
    }
  }
}
```



TEKTON VS JENKINS

기능 비교하면 다음과 같다.

항목	Jenkins	Tekton
개발 주체	Jenkins Community (원래는 Kohsuke Kawaguchi)	Google + CD Foundation
아키텍처	모놀리식(Monolithic), 플러그인 기반	쿠버네티브(Kubernetes Native), CRD 기반
운영 방식	독립 실행형 서버 또는 컨테이너	Kubernetes 클러스터 내에서 실행되는 파이프라인 리소스
파이프라인	Groovy 기반 DSL (Jenkinsfile)	YAML 기반 CRD (Pipeline, Task 등)
확장성	플러그인을 통한 확장 가능, 그러나 종속성 충돌 위험	Kubernetes 자원을 활용한 확장 (Pod, PVC 등)
CI/CD 목적성	CI 중심에서 출발 → CD는 추가 확장	CD를 염두에 둔 구조, 특히 GitOps 및 배포 중심



TEKTON VS JENKINS

항목	Jenkins	Tekton
보안	많은 플러그인은 권한 문제 발생 가능	Kubernetes의 RBAC 기반 보안 정책 활용 가능
사용성	GUI 친화적, 설정 직관적	CLI/yaml 중심으로 비교적 학습 곡선 있음
에이전트 실행	Jenkins 노드로 실행 (Master-Agent 구조)	각 Task가 Pod로 실행, 병렬 처리 용이
상태 저장	Jenkins 자체 DB 사용 (또는 외부 DB)	Kubernetes 리소스로 상태 관리 (ConfigMap, PVC 등)
커뮤니티	매우 활발하고 오래된 커뮤니티	상대적으로 작지만 CNCF 지원 및 문서 지속 증가
적합한 환경	소규모~대규모 전통적인 CI/CD 환경	클라우드 네이티브, 쿠버네티스 중심의 CD 환경
예시 사용처	GitHub Actions과 연동한 기존 서버 중심 파이프라인	GKE, OpenShift 등 쿠버네티스 플랫폼 기반 자동화 파이프라인



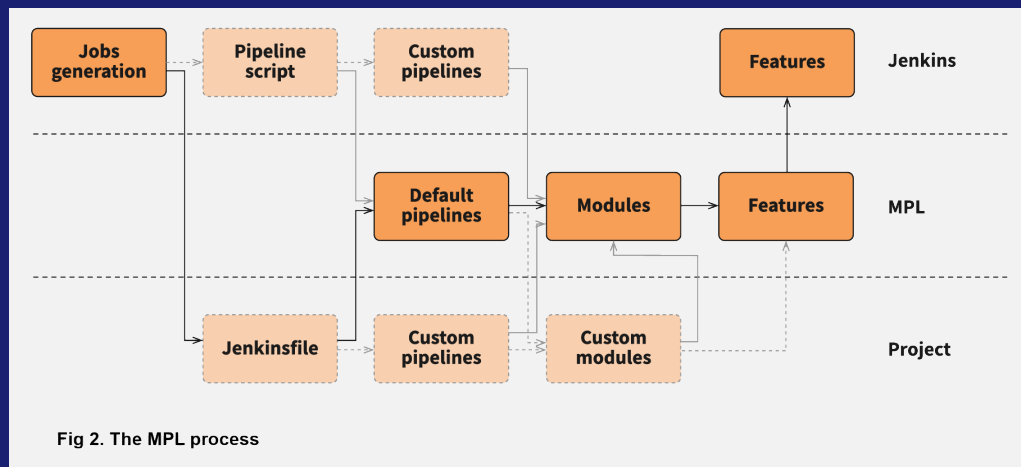
TEKTON VS JENKINS

테크톤과 젠킨스의 제일 큰 차이점은 동작 방식이다.

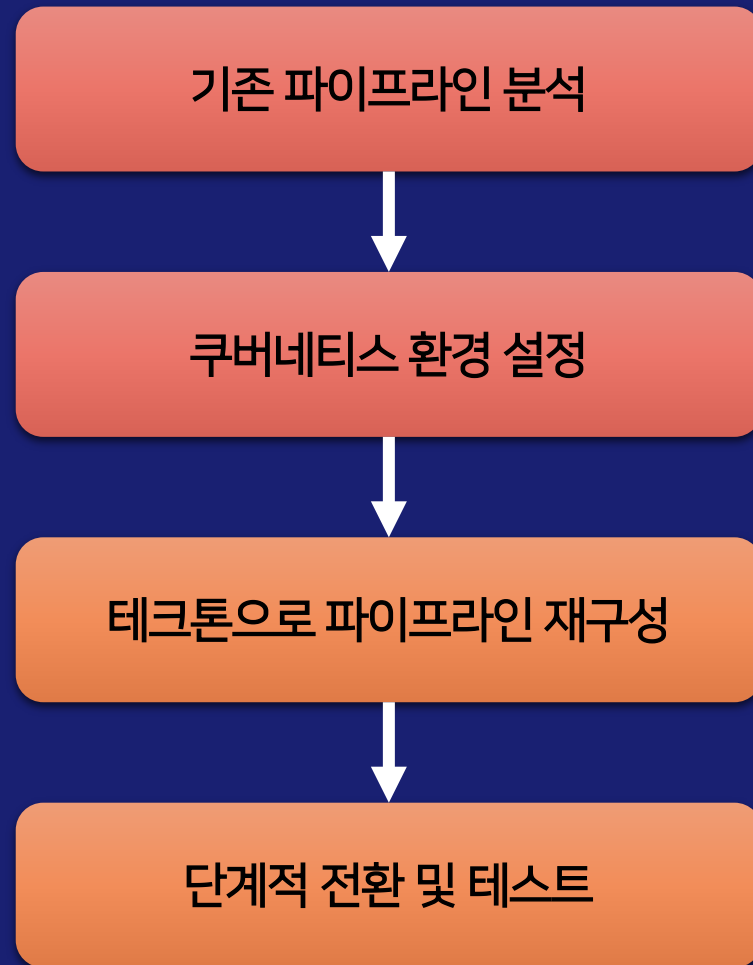
젠킨스는 쿠버네티스와 통합된 형태가 아니다. 본 기능은 자바 빌드용도 사용하다가 쿠버네티스와 통합된 상태이다. 그러한 이유로 젠킨스는 완벽하게 쿠버네티스와 통합이 되지 않는다.

테크톤은 기본적으로 쿠버네티스 클러스터 기반으로 구성 및 작성이 되었다. 젠킨스보다 손쉽게 사용하며, 별도로 관리 모듈을 만들지 않고 사용이 가능하다.

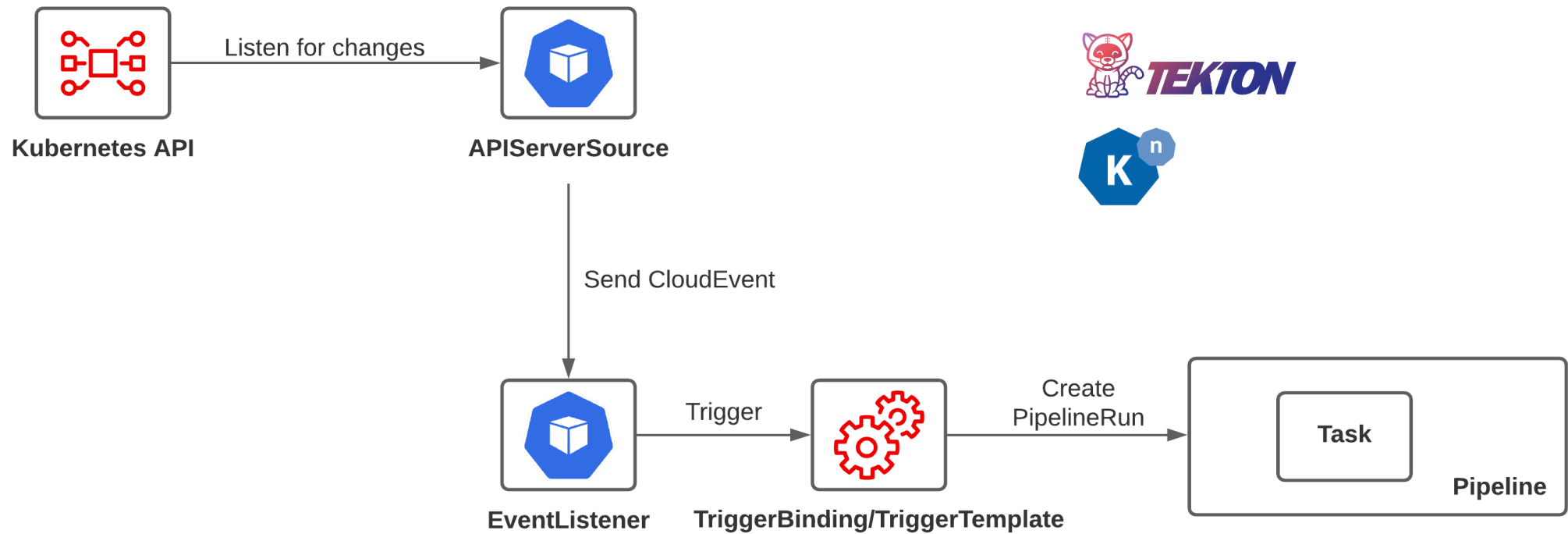
<https://www.jenkins.io/blog/2019/01/08/mpl-modular-pipeline-library/>



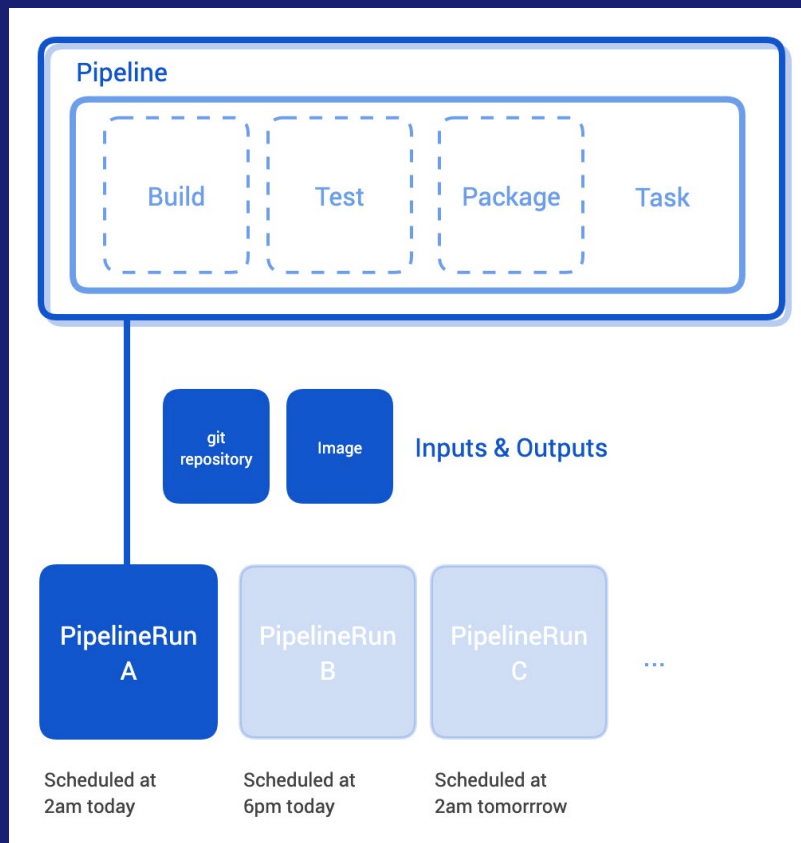
마이그레이션 전략(JKS -> TKN)



ACT IN K8S



TEKTON PIPELINE



클라우드 네이티브에서 CI/CD

CI/CD

Jenkins/Tekton

클라우드 네이티브

클라우드 네이티브는 수많은 컴퓨팅 개념 중 하나이다. 클라우드 네이티브에 제일 큰 차이점은 대다수 오픈소스 기반으로 개념들이 구성이 되어 있으며,

이를 기반으로 Cloud Native Computing Foundation(CNCF)라는 파운데이션을 만들었다. 테크톤은 CD영역에서 포함이 되며, 이를 통해서 K-Native기반의 자동화를 구성한다.



CNCF/K-Native

K-Native는 컨테이너 기반으로 다음과 목적을 가지고 있다.

- **Simpler Abstraction:** YAML 기반으로 CRD 구성이 가능하다.
- **Auto Scaling:** 오토 스케일링 기반으로 0부터 확장 혹은 축소가 가능하다.
- **Progressive Rollouts:** 롤 아웃 상태를 전략에 따라서 구성이 가능하다.
- **Event Integration:** 모든 소스의 이벤트에 대해서 핸들링이 가능하다.
- **Handle Events:** 작업 이벤트를 트리거를 통해서 관리한다.
- **Pluggable:** 쿠버네티스에 확장 기능 추가가 가능하다.



CR/CRD

CR

CR, **Custom Resource**라고 부르며, 쿠버네티스 API를 통해서 생성이 된다.

대표적인 자원은 **Pod**, **ConfigMap** 그리고 **Secrets**이 있다. CR자원은 별도로 설치 및 구성이 필요 없으며 쿠버네티스 클러스터에서 지원 및 제공한다.

CRD

CRD, **Custom Resource Definition**의 약자이다. 쿠버네티스 네이티브 API가 아닌, 추가적으로 설치한 애플리케이션에 자원을 생성 시, CRD를 통해서 구성한다. 테크톤도 CRD를 통해서 자원을 생성 및 구성한다.



테크톤 자원

Stepping

Tasks

STEP

기본자원

STEP

단계(Step)은 제일 기본적인 유닛이며, 이를 기반으로 파이프라인 구성이 된다. 최소 한 개의 작업이라도 단계에 구성이 되어야 한다. 이를 통해서 CI/CD 작업 수행 단계를 효율적으로 구성 및 운영이 가능하다.

각 단계에서는 명령어를 통해서 작업이 수행이 되는데, 예를 들어서 이미지 빌드를 위해서 소스코드를 내려받기 후 컴파일 과정이 필요하다면, 이 부분을 단계에 명시한다. 각 단계에는 일반적으로 명령어를 통해서 어떠한 작업을 수행 할지 명시한다. 자세한 내용은 뒤에서 더 다룬다.

```
spec:
```

```
  steps:
```

```
    - image: quay.io/centos/centos
```

```
      command:
```

```
        - /bin/bash
```

```
        - -c
```

```
        - echo "Hello World"
```

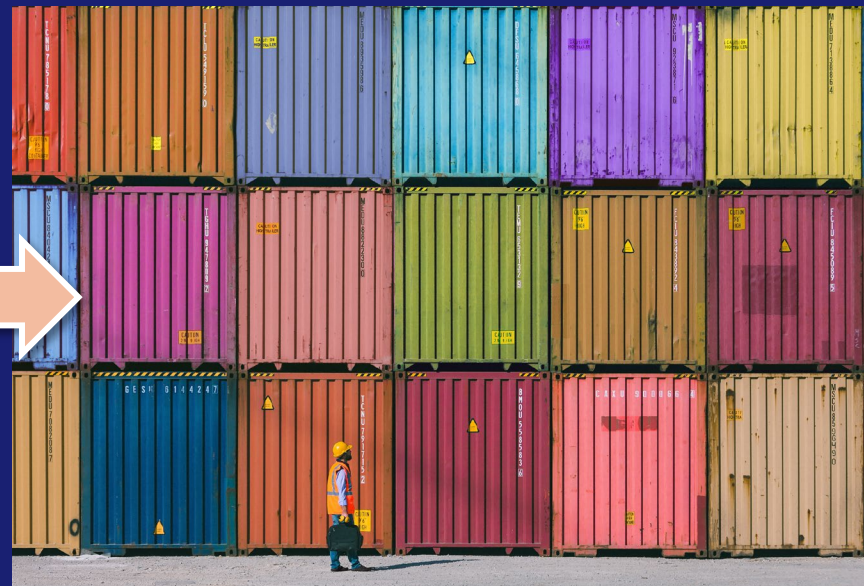


STEP

spec:

steps:

- name:



TASK, PIPELINE

기본 자원

TASK

파이프 라인은 작업의 순서를 정하는 자원이다.

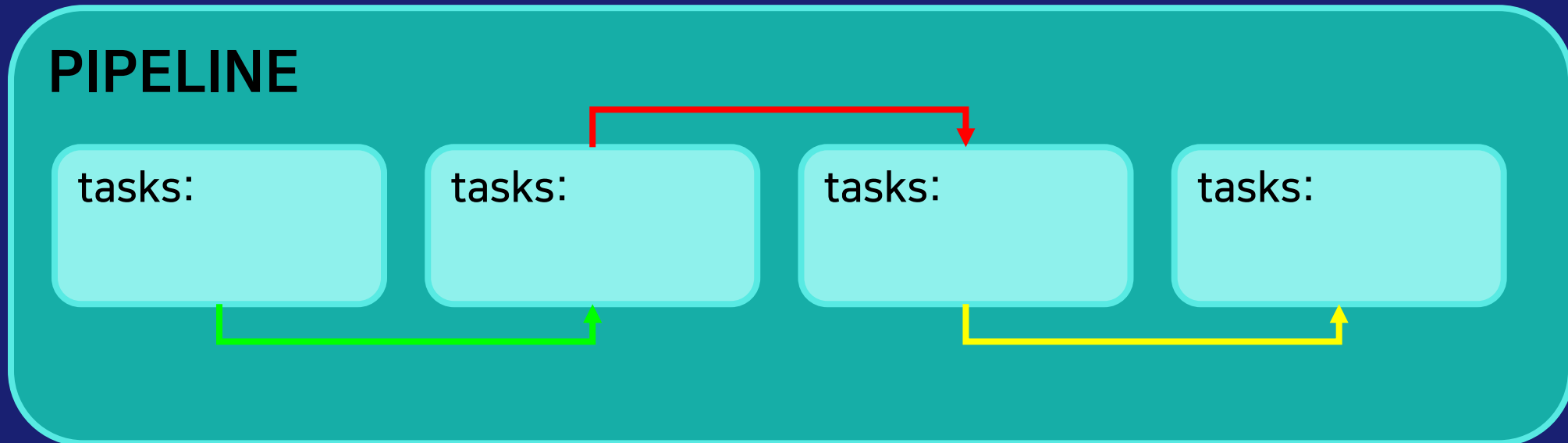
쉽게 생각하면 GW-BASIC처럼, 각각 작업에 순서를 걸어서 순차적으로 작업을 수행한다. 파이프 라인은 추상적인 자원이며, 이 자원은 기본적으로 태스크를 감싸고 있다.



PIPELINE

모든 스텝 및 태스크에는 작업 순서가 있기 때문에, 작성된 순서대로 작업이 하나씩 수행이 된다.

이러한 형태를 가지고 있는 자동화 도구는 앤서블/솔트와 같은 도구들이 있다. 이들은 테크톤과 동일하게 **YAML**기반으로 작업을 작성 및 구성하게 된다.



PIPELINE

아래는 파이프라인 생성 YAML코드 예제. 파이프라인은 반드시 어떤 작업(task)를 수행할지 명시해야 한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: first-pipeline
spec:
  tasks:
  - name: first
    taskRef:
      name: first-task
```

EOF



TASK

태스크(tasks)는 파이프라인에 안에서는 작업에 대한 격리를 한다.

여기서 말하는 **작업은 단계(step)**를 이야기 하며, 각각 단계들은 태스크에서 수행이 된다. 각 태스크는 단계들의 **순서(sequence)**를 가지고 있으며, 이를 통해서 하나의 작업을 통해서 모든 작업을 수행한다.

쿠버네티스 클러스터에서 Pod를 생성하기 위해서 이미지 작업 및 Pod생성과 같은 과정이 필요하다. 이러한 부분을 각각 단계로 구성하여 작업을 수행 후, 최종적으로 API를 통해서 Pod생성 및 PV/PVC와 같은 자원을 구성 및 연결을 수행한다.

이러한 **작업 단계를 모아서 동작하는 영역이 파이프라인**이며, **작업(task)**는 **단계(step)**에 명시된 작업들을 분리 및 격리하여 수행 할 수 있도록 한다. 작업을 구성하면, 이들은 **파이프 라인**을 통해서 구성 및 실행이 된다.



TASK YAML

아래는 간단하게 태스크(task)를 생성하는 YAML이다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: first-task
spec:
  steps:
  - name: first-task
    image: quay.io/centos/centos
    command:
      - echo "hello world"
```

EOF



생성 자원 확인

생성된 자원은 아래와 같이 명령어로 확인이 가능하다.

```
tkn pipeline list
tkn task list
tkn pipeline start first-pipeline
```

```
tkn pipeline list
```

NAME	AGE	LAST RUN	STARTED
first-pipeline	14 minutes ago	first-pipeline-run-86x7v	19 seconds ago



테크톤과 인공지능

SLM

진행 전 잠깐!

테크톤 YAML은 버전에 따라서 사라지는 문법 및 구조가 종종 발생한다. 이러한 이유로, 테크톤에서 작성하는 YAML은 상황에 따라서 매우 불편하고 귀찮은 부분이 있다.

이러한 이유로, 코드 작성 시 대형 LLM이 아닌 코드 생성에 특화된 SLM(Small Language Model)을 활용하여 뼈대 코드(Skeleton Code)를 생성한 뒤 수정하는 방식을 권장한다. 하지만, 테크톤에서 제공하는 예제 코드는 많은 영역과 예제를 제공하지 않기 때문에 인공지능 시스템을 사용해서 코드 작성을 도움을 받는다.



SLM용으로 사용하기 좋은 모델

SLM용으로 사용하기 위해서 huggingface에서 다음 모델을 다운로드 받는다.

```
dnf install -y git python3-pip podman jq
python3 -m pip install -U "huggingface_hub[cli]"
hf auth login
hf download Qwen/Qwen2.5-Coder-1.5B-Instruct \
  --local-dir qwen2.5-coder-1.5b-instruct
curl -o slm-tkn.sh <code>lab</code>
```



SLM RUNTIME 설치(CPU 기반)

여기서는 KIKI Project에서 제공하는 명령어를 통해서 테크톤 YAML 파일을 생성한다. 우리가 사용하는 환경은 GPU가 없는 CPU 전용 환경이기 때문에, CPU 기반으로 구성한다.

간단하게 구성하기 때문에 다음과 같이 llama.cpp를 통해서 구성 및 설치한다.

```
mkdir -p /opt/models/  
curl -o /opt/models/Qwen2.5-Coder-1.5B-Instruct-Q2_K.gguf  
http://10.0.0.250/Qwen2.5-Coder-1.5B-Instruct-Q2_K.gguf  
podman run -d --replace --name slm-tekton -p 8080:8080 \  
    -v /opt/models:/models:ro \  
    --cpus=4 --memory=12g ghcr.io/ggml-org/llama.cpp:server \  
    --model /models/Qwen2.5-Coder-1.5B-Instruct-Q2_K.gguf \  
    --host 0.0.0.0 --port 8080 \  
    --ctx-size 8192 --n-gpu-layers 0  
podman container ls
```



SLM RUNTIME 테스트

다음과 같은 명령어 코드 생성이 가능하다. 정상적으로 동작할 경우, Tekton Task YAML 코드만 출력된다.

```
# curl http://localhost:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "local",
  "messages": [
    {
      "role": "system",
      "content": "You are a Tekton expert. Generate valid Tekton YAML only. No explanation."
    },
    {
      "role": "user",
      "content": "Create a simple Tekton Task named hello-task that echoes hello."
    }
  ]
}'
```



코드 생성 래퍼

아래 스크립트를 사용하여 손쉽게 사용이 가능하다.

```
chmod +x slm-tekton.sh
slm-tekton.sh --prompt "Create a Tekton Task named hello-task that echoes
hello." --out hello-task.yaml
cat > prompt.txt <<'EOF'
Create a Tekton Task named build-image.
- steps: git clone, build
- use workspace: shared-data
Output YAML only.
EOF
slm-tekton.sh --prompt-file prompt.txt --out build-task.yaml
```



사용 방법

강사와 함께 진행 한다.



기본 자원

Stepping

Task

Pipe

STEP 설명

단계(스텝)은 특정 작업을 수행하는 부분이다. 단계는 다음과 같이 작업을 수행한다.

1. 소스코드 내려받기
2. 컴파일 및 명령어 수행
3. 프로그램 패키징

총 3단계를 통해서 작업을 한다. 이미지가 비공개 저장소에 있는 경우, **ImagePullSecret**를 통해서 내려받기가 가능하다. 작성 방법은 아래와 같이 작성이 가능하다.



STEP 예제

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-task-1
spec:
  steps:
    - image: quay.io/centos/centos:stream9
      command:
        - /bin/bash
        - -c
        - echo "Hello World"
```

EOF



STEP 예제 확인

생성한 **demo-task-1**를 아래와 같이 확인한다.

```
tkn task ls
```

NAME	DESCRIPTION	AGE
demo-task-1		7 seconds ago
hello		3 minutes ago

```
tkn task start demo-task-1
```

```
tkn task start demo-task-1 --showlog
```



STEP FOR MULTI STEP

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-multi-step
spec:
  steps:
    - name: first
      image: quay.io/centos/centos:stream9
      command:
        - /bin/bash
        - -c
        - echo "First step"
```



STEP FOR MULTI STEP

```
- name: second
  image: quay.io/centos/centos:stream9
  command:
    - /bin/bash
    - -c
    - echo "Second step"
```

EOF

```
tkn task list
```

```
tkn task start demo-multi-step
```

```
tkn taskrun logs demo-multi-step-run-hlvvgx -f
```



STEP IMAGE

단계 구성 시, 사용하는 컨테이너 이미지는 사용자가 원하는 이미지로 변경이 가능하다. 여기서는 일반적으로 많이 사용하는 **centos** 이미지 기반으로 구성하였다. 이미지는 최소 한 개가 구성 및 선언이 되어야 한다.

문법은 아래와 같이 선언이 된다.

현재 연습 예제에서는 centos 기반으로 테스트를 수행 및 진행한다. 만약, 다른 배포판 사용을 원하는 경우, 다른 컨테이너 이미지를 사용하여도 된다.

```
- name: second  
  image: quay.io/centos/centos
```



STEP 실행 및 상태 확인

올바르게 실행이 되었는지 아래 명령어로 확인한다.

```
tkn task list
```

```
tkn task start demo-multi-step --showlog
```

```
tkn task start demo-multi-step --showlog --no-color
```

```
kubectl get tasks
```

```
kubectl get taskruns
```



STEP 스크립트 사용

일반적으로 스크립트를 임베디드 형태로 사용 시, 아래와 같이 사용한다.

```
- name: step-in-script
  image: quay.io/centos/centos:stream9
  script: |
    #!/usr/bin/env bash
    echo "Install a package"
    dnf install httpd -y
    dnf clean all
    echo "All commands ran!"
```



STEP IN SCRIPT

위의 내용을 실제로 적용하면 다음과 같이 적용이 된다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1beta1  
kind: Task  
metadata:  
  name: demo-step-script  
spec:
```



STEP IN SCRIPT

위의 내용 계속 이어서...

```
steps:  
- image: quay.io/centos/centos:stream9  
  script: |  
    #!/usr/bin/env bash  
    echo "Install a package"  
    dnf install httpd -y  
    dnf clean all  
    echo "All commands ran!"
```

EOF



STEP IN SCRIPT

아래 명령어 적용 및 확인한다.

```
tkn task list  
tkn task start demo-step-script --showlog
```



TASK PARAMETER

앞에서 간단하게 단계와 작업을 동시에 작성 및 실행 하였다. 하지만, 매번 작업을 구성할 때마다 단계에 들어가는 값을 변경할 수 없기 때문에, 재사용을 위해서 파라미터 형식으로 변경한다.

파라미터, 즉 변수 형태로 하는 경우, **param**항목에서만 변경하면서 사용이 가능하다. 좀 더 효율적으로 운영 및 구성이 가능하다.

위와 같이 값의 이름, 그리고 유형을 적어주면, 쉘 텍스트 입력 받는 형식과 비슷하게 값을 읽어온다. 예로, 앞에서 사용하였던 명령어 실행을 예로 든다.

```
params:  
  - name: username  
    type: string
```



TASK PARAMETER

아래와 같이 작성하면, 변수를 통해서 좀 더 효율적으로 코드 사용이 가능하다.

command:

- /bin/bash

- -c

- echo "Hello \${params.who}"



TASK PARAMETER 작성 1

위의 두 개 예제를 가지고 다음과 같이 테크톤 작업형식으로 작성이 가능하다. 아래 슬라이드처럼 파일을 작성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-task-param
spec:
  params:
  - name: username
    type: string
```



TASK PARAMETER 작성 2

위의 내용 계속 이어서...

steps:

- image: quay.io/centos/centos:stream9
- command:
- /bin/bash
 - -c
 - echo "Hello \$(params.username)"



TASK PARAMETER 확인

적용 후, 실행하면 다음과 같이 실행 및 확인이 된다.

```
tkn task list
```

```
tkn task start demo-task-param --showlog
```

```
? Value for param 'username' of type 'string'? tang
```

```
TaskRun started: demo-task-param-run-9rpcz
```

```
Waiting for logs to be available ...
```

```
[unnamed-0] Hello tang
```

```
tkn task start demo-task-param --showlog -p username=tang
```



TASK ARRAY PARAMETER 적용 1

파라미터를 배열로 처리가 가능하다. 다만, 좀 더 복잡하게 코드를 구성해야 한다. 기존 코드에 아래와 같이 파일 이름을 변경 후 작성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-task-param-array
spec:
  params:
    - name: users
      type: array
```



TASK ARRAY PARAMETER 적용 2

파라미터를 배열로 처리가 가능하다. 다만, 좀 더 복잡하게 코드를 구성해야 한다. 기존 코드에 아래와 같이 파일 이름을 변경 후 작성한다.

steps:

- name: list-users
- image: quay.io/centos/centos:stream9
- args:
 - \$(params.users[*])
- command:
 - /bin/bash
 - -c
 - for ((i=1;i<=\$#;i++)); do echo "\$#" "\$i" "\${!i}"; done

EOF

배열로 사용자 값을 받아온다.

계
속



TASK PARAMETER ARRAY 적용 및 실행

테크톤에 작업 등록 후 작업을 수행한다.

```
tkn task list  
tkn task start demo-task-param-array --showlog --use-param-defaults  
test1, test2, test3
```



TASK DEFAULT PARAMETER 선언 및 생성 1

파라미터에 기본값 설정이 필요한 경우 `default`라는 변수 키워드를 사용한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-task-param-default
spec:
  params:
  - name: users
    type: array
    default:
      - test1
      - test2
      - test3
```



TASK DEFAULT PARAMETER 선언 및 생성 2

앞에 내용 계속 이어서...

steps:

- name: list-users
- image: quay.io/centos/centos:stream9
- args:
 - \$(params.users[*])
- command:
 - /bin/bash
 - -c
 - for ((i=0;i≤\$#;i++)); do echo "\$#" "\$i" "\${!i}"; done

EOF



TASK DEFAULT PARAMETER 확인 및 실행

기본 변수 값을 쿠버네티스에 등록 후 실행한다.

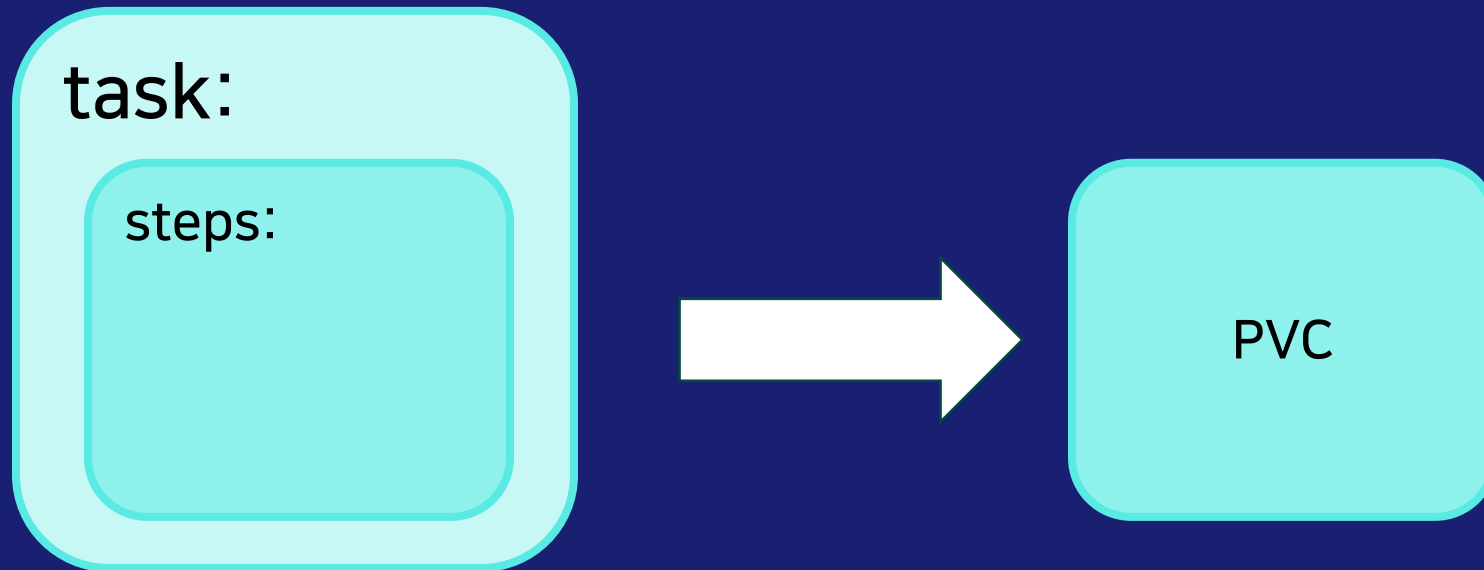
```
tkn task list
```

```
tkn task start demo-task-param-default --use-param-defaults --showlog
```



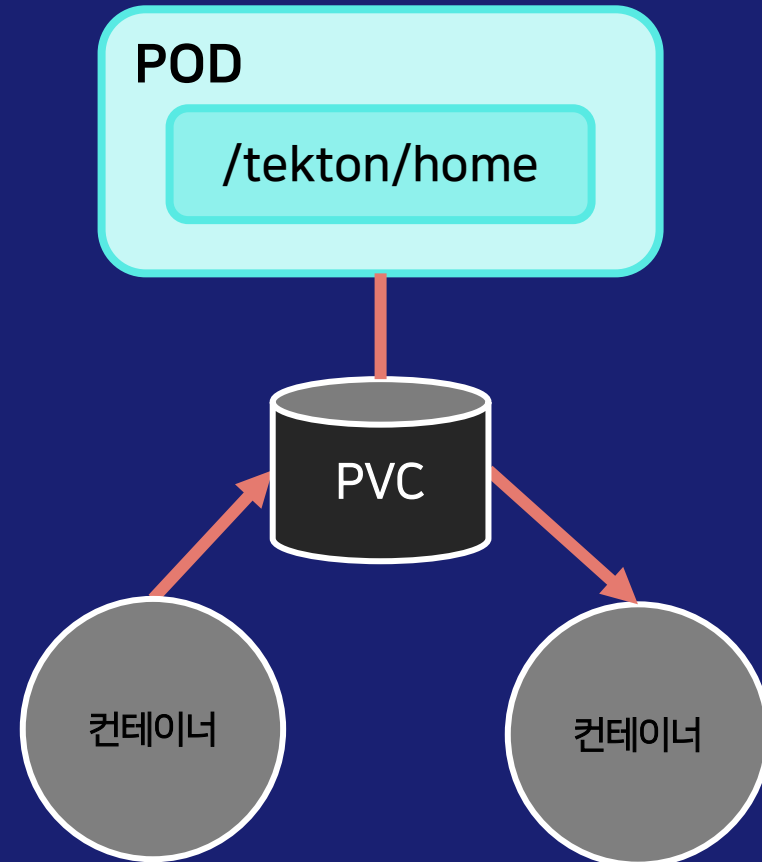
공유 데이터 설명

테크톤이 동작하면 컨테이너가 하나의 **포드(Pod)**에서 동작하기 때문에, 데이터 공유가 가능하다. 데이터를 공유 하기 위해서 쿠버네티스의 **PV/PVC**를 통해서 공유가 가능하다.



테크톤 공유 디렉터리

테크톤에서 제공하는 고유한 디렉터리를 사용하여 POD에서 컨테이너간 데이터 공유가 가능하다.
다음 챕터에서 이 부분에 대해서 자세히 다룬다.



기본 자원 소개

share directory

result

공유 데이터 설명

테크톤은 특정 디렉터리 위치에 데이터를 임시적 혹은 영구적으로 공유가 가능하다.

테크톤 컨테이너가 실행 및 동작하면 다음과 같은 디렉터리를 생성하여 테크톤에 연결한다. 예약된 디렉터리 및 변수는 다음과 같다.



공유 데이터 설명

공유 데이터 위치는 다음과 같다.

디렉터리 경로	용도 설명	접근 주체
/workspaces	Task 및 Step 간 공유 작업 공간 (Pipeline level에서 연결된 워크스페이스)	사용자 정의
/tekton/home	컨테이너의 기본 작업 디렉터리 (HOME). git clone 등의 작업 경로로 사용	Tekton Controller
/tekton/creds	인증 정보가 저장되는 디렉터리. git, docker 등 Secret/ServiceAccount 기반 인증 정보 주입 위치	Tekton 시스템
/tekton/results	Step 실행 결과(예: 출력 값)를 저장하는 디렉터리. results 필드와 연동됨	Tekton 시스템



공유 데이터 설명

공유 데이터 위치는 다음과 같다.

디렉터리 경로	용도 설명	접근 주체
/tekton/run	TaskRun 실행 정보 (metadata 등)가 저장되는 임시 경로 (일부 구현체에서 사용)	Tekton 내부용
/tekton/tools	일부 도구/실행 파일들(executable)이나 CLI 바이너리가 임시로 설치되는 위치	Tekton 시스템
/tekton/tmp	임시 파일 저장소. 스크립트 작업 중 중간 파일 생성 시 활용됨	각 Step



공유 데이터(HOME) 확인하기 1

간단하게 기본 테크톤 공유 디렉터리를 생성한다. 아래 처럼 예제 파일을 구성 및 생성한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1beta1  
kind: Task  
metadata:  
  name: tekton-share-home
```



공유 데이터(HOME) 확인하기 2

여기서는 공유 디렉토리를 선언하지 않았다. 그러므로 각각의 단계 데이터만 출력한다.

```
spec:
```

```
  steps:
```

```
    - name: write
```

```
      image: quay.io/centos/centos:stream9
```

```
      script: |
```

```
        echo "location is $(pwd)"
```

```
        echo ~
```

```
        echo ~Tekton
```

```
        touch /tekton/home/tkn-message.txt
```



공유 데이터(HOME) 확인하기 3

```
- name: read
  image: quay.io/centos/centos:stream9
  script: |
    echo "Listing /tekton/home"
    ls -lR /tekton/home
- name: tree
  image: quay.io/centos/centos:stream9
  script: |
    dnf install tree -y
    tree -L 2 /tekton/
    tree -L 2 /workspace/
```



공유 데이터(HOME) 확인하기 3

```
kubectl apply -f tekton-share-home.yaml
```

```
tkn task list
```

```
tkn task start tekton-share-home --showlog
```



출력되는 내용들은 각기 다르게 보인다.
문제가 없으면 옆에 그림처럼 디렉터리가
출력이 된다.



참고사항

위의 기능은 최신 테크톤 버전에서는 동작하지 않는다. 자세한 정보는 아래 링크를 참고한다.

<https://github.com/tektoncd/pipeline/pull/3878>

테크톤 포드에서는 더 이상 앞에서 언급한 디렉터리 **"/tekton/home"**, **"/workspace"**는 변수를 통해서 접근이 가능하다.

<https://github.com/tektoncd/pipeline/blob/main/docs/variables.md>



공유 데이터(RESET)

특정 작업이 수행 후, 결과값에 대해서 저장을 하기 위해서 result라는 지시자 사용이 가능하다.

1. 결과(result)를 사용하면, 완료된 작업의 내용을 /tekton/results에서 접근 및 사용이 가능하다.
2. 결과(result)는 쿠버네티스의 PV/PVC를 사용하지 않는다.
3. 경량 데이터를 전달이 주요 목적이다

이전에 사용한 내용을 응용하여 결과(result)를 사용하도록 한다.



TEKTON RESULTL 적용 및 확인 1

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1beta1  
kind: Task  
metadata:  
  name: tekton-result  
spec:  
  results:  
    - name: welcome  
      description: welcome message
```



TEKTON RESULTL 적용 및 확인 2

steps:

- name: **write**
image: quay.io/centos/centos:stream9
command:
 - /bin/bash
- args:
 - "-c"
 - echo "Welcome to Tekton world" > \$(results.welcome.path)



TEKTON RESULTL 적용 및 확인 3

```
- name: read
  image: quay.io/centos/centos:stream9
  command:
    - /bin/bash
  args:
    - "-c"
    - cat $(results.welcome.path)
```

```
tkn task list
```

```
tkn task start tekton-result --showlog
```



KUBERNETES VOLUME FOR TKN 설명

쿠버네티스에서 사용하는 볼륨 자원, `secret`, `configmap` 그리고 `Persistent Volume`, `Persistent Volume Claim` 전부 사용이 가능하다.

여기에서는 간단하게 쿠버네티스 볼륨 자원들을 테크톤에서 활용해보도록 한다. 더 많은 자원은 뒤에서 추후 활용 및 다루기로 한다.

다루는 자원은 다음과 같다.

1. `secret`
2. `configmap`
3. `PV/PVC`



TEKTON CONFIGMAP 연결 1

테크톤에서 ConfigMap에 접근 할 수 있도록 아래와 같이 테스트용 CM자원을 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-list
data:
  user1: helix
  user2: tang
  user3: suse
EOF
```



TEKTON CONFIGMAP 연결 2

테크톤에서 CM에 접근이 가능한지 아래 예제로 테스트한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: tekton-demo-configmap
spec:
  volumes:
    - name: users
      configmap:
        name: user-list
```



TEKTON CONFIGMAP 연결 3

steps:

```
- name: username-list
  image: quay.io/centos/centos
  volumeMounts:
    - name: users
      mountPath: /var/username-list
  script: |
    echo "$(cat /var/username-list/user1) is normal user"
    echo "$(cat /var/username-list/user2) is root user"
    echo "$(cat /var/username-list/user3) is admin user"
```

EOF



TEKTON CONFIGMAP 결과 확인

```
tkn task list  
tkn task start tekton-demo-configmap --showlog
```



디버깅

테크톤

DIGGING and DIGGING 설명



디버깅 사용 방법

디버깅 위한 명령어는 다음과 같이 지원 및 제공한다.

```
tkn task start tekton-configmap --showlog  
kubectl run --image=quay.io/centos/centos:stream9  
kubectl get taskrun tekton-configmap-run-<ID> -o yaml  
tkn taskrun list
```



테크톤에서 디버깅 1

아래와 같이 YAML파일 생성 후 다음과 같이 명령어를 통해서 디버깅을 시도한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: failforfail
spec:
  steps:
  - image: quay.io/dollar/dollar
    command:
      - echo "Hello failure"
EOF
```



디버깅 적용 및 확인

아래와 같이 YAML파일 생성 후 다음과 같이 명령어를 통해서 디버깅을 시도한다.

```
kubectl apply -f failforfail.yaml  
tkn task start failforfail --showlog  
kubectl get tr  
kubectl get tr failforfail-run-<ID> -o yaml
```



파이프라인

소개

파이프라인 소개

파이프라인은 앞서 사용하였던 **단계(Step)**, **작업(Task)**를 조합하여 실행한다.

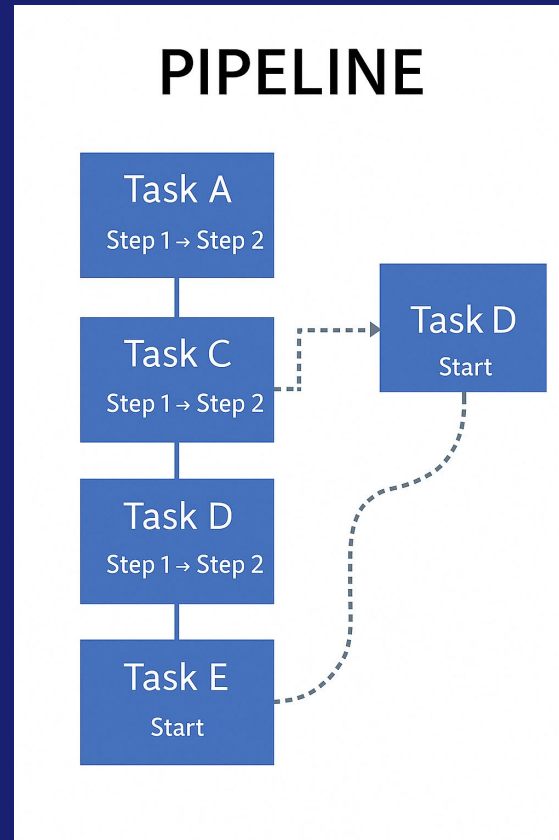
또파이프라인은 쿠버네티스에서 CI/CD인터페이스를 통해서 작업을 수행 시, 테크톤은 파이프라인을 호출하여 작업을 실행 및 수행한다. 예를 들어서 이전에 생성한 작업에 **"복제"**, **"컴파일"**, **"검증"**과 같은 작업이 있었다면, 이를 태스크를 통해서 작업을 수행한다.

태스크의 주요 목적은 최대한 자원을 재사용과 손쉽게 작업을 사용자가 원하는 순서대로 구성하는 게 주요 목적이다. 파이프 라인도 YAML기반으로 작성이 된다.

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: demo-pipeline
labels:
  key: value
spec:
```



소개



파이프라인 생성

파이프라인

파이프라인에 사용할 태스크 생성 1

이전에 사용했던 내용을 다시 반복한다. 작업 생성 후 파이프라인을 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: hello-world
spec:
  steps:
    - name: echo
      image: alpine
      script: |
        echo "Hello Tekton!"
```



계
속

파이프라인에 사용할 태스크 생성 2

```
---
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: hello-pipeline
spec:
  tasks:
    - name: say-hello
      taskRef:
        name: hello-world
EOF
```



파이프 라인 생성 1

간단하게 **step**, **task**를 생성 후, 파이프 라인으로 작업을 호출한다.

위의 코드는 쉽게 구성 및 구현하기 위해서 task와 pipeline를 한 파일에 동시 선언 하였다. 쉽게 실행하기 위해서 **pipelinerun**이라는 자원을 통해서 실행이 가능하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  name: hello-pipelinerun
spec:
  pipelineRef:
    name: hello-pipeline
EOF
```



파이프 라인 실행 및 디버깅 1

아래와 같이 확인 및 디버깅이 가능하다.

```
tkn pipelinerun list
```

NAME	STARTED	DURATION	STATUS
hello-pipelinerun	2 minutes ago	5s	Succeeded

```
tkn pipelinerun describe hello-pipelinerun
```

Name: hello-pipelinerun

Status: Succeeded

StartTime: 2 minutes ago

Duration: 5s

Tasks:

NAME	STATUS
------	--------

• say-hello	Succeeded
-------------	-----------



파이프 라인 실행 및 디버깅 2

아래와 같이 확인 및 디버깅이 가능하다.

```
tkn pipelinerun logs hello-pipelinerun -f  
[say-hello : echo] Hello Tekton!
```



파이프 라인 디자인

이번에는 앞에 내용에 **깃/파일** 목록 출력 관련된 작업을 출력하는 내용이다. 테크톤에서 제일 많이 하는 작업은 앞서 이야기 하였지만 다음과 같은 순서로 진행한다.

1. `git clone`
2. `build`
3. `package`
4. `application image push`
5. `deploy to kubernetes cluster`

이 중, **1번과 2번**만 파이프라인으로 수행한다.



태스크 생성 1

먼저, 테크톤 컨테이너를 통해서 소스코드를 클론 받는다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1  
kind: Task  
metadata:  
  name: fetch-repo
```



태스크 생성 2

```
spec:
  workspaces:
    - name: shared
  steps:
    - name: clone
      image: alpine/git
      script: |
        git clone https://github.com/tektoncd/pipeline.git
        /workspace/shared
EOF
```



파일목록 확인 작업 1

테크톤 작업을 통해서 컨테이너의 파일 목록을 확인한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1  
kind: Task  
metadata:  
  name: list-files
```



파일목록 확인 작업 2

```
spec:
  workspaces:
    - name: shared
  steps:
    - name: ls
      image: alpine
      script: |
        ls -al /workspace/shared
EOF
```



파이프라인 구성 1

위의 작업을 파이프 라인으로 통합한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: git-pipeline
spec:
  workspaces:
    - name: shared
```



파이프라인 구성 2

앞에 내용 계속 이어서...

```
tasks:
  - name: fetch
    taskRef:
      name: fetch-repo
    workspaces:
      - name: shared
        workspace: shared
```



파이프라인 구성 3

```
- name: list
  runAfter: ["fetch"]
  taskRef:
    name: list-files
  workspaces:
    - name: shared
      workspace: shared
```

EOF



결과 확인 1

적용 후 아래와 같이 파이프 라인 실행 결과를 확인한다.

```
tkn pipelinerun list
```

NAME	STARTED	DURATION	STATUS
git-pipelinerun	10 seconds ago	8s	Succeeded



결과 확인 2

```
tkn pipelinerun describe git-pipelinerun
```

```
Name:      git-pipelinerun
```

```
Namespace: default
```

```
Status:    Succeeded
```

```
StartTime: 2025-04-20T16:15:00Z
```

```
Completion: 2025-04-20T16:15:08Z
```

```
Duration:   8s
```

```
Tasks:
```

NAME	TASK NAME	STARTED	DURATION	STATUS
• fetch	fetch-repo	10 seconds ago	5s	Succeeded
• list	list-files	5 seconds ago	3s	Succeeded



결과 확인 3

```
tkn pipelinerun logs git-pipelinerun -f
[fetch : clone] Cloning into '/workspace/shared' ...
[fetch : clone] remote: Enumerating objects: 10, done.
[fetch : clone] remote: Counting objects: 100% (10/10), done.
[fetch : clone] remote: Compressing objects: 100% (8/8), done.
[fetch : clone] Receiving objects: 100% (10/10), done.

[list : ls] total 24
[list : ls] drwxr-xr-x    3 root    root        4096 Apr 20
16:15 .github
[list : ls] -rw-r--r--    1 root    root        1234 Apr 20 16:15
README.md
```



파이프라인 파라미터

파이프라인

파이프라인 파라미터 구성

파이프라인은 파라미터 기반으로 동작이 가능하다. 다음과 같은 조건으로 동작한다.

1. Pipeline 실행 시 외부에서 값을 주입
2. Pipeline과 Task의 동작을 유연하게 제어하는 변수

이를 통해서 재활용을 높여서 YAML 수정하지 않고 CI/CD 인터페이스를 통해서 반복적으로 사용이 가능하도록 한다.

항목	설명
name	파라미터 이름
type	string//object
description	파라미터 설명 (문서화 중요)
default	기본값 (선택)



태스크 생성 1

테스트를 위해서 태스크부터 생성한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1  
kind: Task  
metadata:  
  name: generate-message  
spec:  
  results:  
    - name: output  
      description: message text
```



태스크 생성 2

```
steps:
```

```
  - name: echo
```

```
    image: alpine
```

```
    script: |
```

```
      echo -n "메시지 전달 완료!" > $(results.output.path)
```

```
EOF
```



출력 태스크 생성 1

태스크 메시지 출력 부분을 아래와 같이 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: print-message
spec:
  params:
    - name: input
```



출력 태스크 생성 1

```
steps:
```

```
  - name: echo
```

```
    image: alpine
```

```
    script: |
```

```
      echo "전달된 메시지: ${params.input}"
```

```
EOF
```



파이프라인으로 연결 1

모든 작업을 파이프 라인으로 통합한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1  
kind: Pipeline  
metadata:  
  name: result-pipeline  
spec:
```



파이프라인으로 연결 2

```
tasks:  
  - name: generate  
    taskRef:  
      name: generate-message  
  - name: print  
    taskRef:  
      name: print-message  
  params:  
    - name: input  
      value: $(tasks.generate.results.output)
```

EOF



파이프라인 연결 확인 1

올바르게 파이프라인 실행에 등록이 되었는지 확인한다.

```
tkn pipelinerun list
```

NAME	STARTED	DURATION	STATUS
result-pipelinerun	5 seconds ago	4s	Succeeded



파이프라인 연결 확인 2

```
tkn pipelinerun describe result-pipelinerun
```

```
Name:      result-pipelinerun
Namespace:  default
Status:     Succeeded
StartTime:  2025-04-20T16:30:00Z
Completion: 2025-04-20T16:30:04Z
Duration:   4s
```

Tasks:

NAME	TASK NAME	STARTED	DURATION	STATUS
• generate	generate-message	4s ago	2s	Succeeded
• print	print-message	2s ago	2s	Succeeded



파이프라인 연결 확인 3

```
tkn pipelinerun logs result-pipelinerun -f  
[generate : echo] 메시지 전달 완료!  
[print : echo] 전달된 메시지: 메시지 전달 완료!
```



테크톤에서 빌드

파이프라인

테크톤에서 빌드

이번에는 파이프라인 기반으로 빌드 하는 방법에 대해서 학습한다.

여기서는 사용하는 언어 예제는 GO언어로 기반으로 진행한다. 각 언어 및 프레임워크에 따라서 지원 방법이 다르기 때문에, 다른 언어를 사용하는 경우 각각의 방식에 맞게 구성한다.



빌드용 태스크 구성 1

작업을 수행하기 위해서, TaskRun으로 바로 작업 수행이 가능하도록 한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: TaskRun
metadata:
  name: git-clone-run
spec:
  taskRef:
    name: git-clone
    bundle: gcr.io/tekton-releases/catalog/upstream/git-clone:0.
```



빌드용 태스크 구성 2

params:

- name: url
value: "https://github.com/gookhyun/go-hello-world.git"
- name: revision
value: "main"
- name: subdirectory
value: ""
- name: deleteExisting
value: "true"



빌드용 태스크 구성 3

```
workspaces:  
  - name: output  
    persistentVolumeClaim:  
      claimName: go-source-pvc
```

빌드를 위해서는 최소 한 개의 빌드 저장소가 필요하다.
여기서 생성된 바이너리 기반으로 배포한다.



빌드용 태스크 구성 4

```
workspaces:  
  - name: output  
    persistentVolumeClaim:  
      claimName: go-source-pvc
```

EOF



컴파일 태스크 구성 1

컴파일 테스트를 위해서 아래와 같이 작성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: go-build
spec:
  workspaces:
    - name: source
```



컴파일 태스크 구성 2

steps:

```
- name: build
  image: golang:1.20
  workingDir: $(workspaces.source.path)
  script: |
    #!/bin/sh
    go mod tidy
    go build -o hello-app
```

EOF



빌드 파이프 라인 구성 1

빌드 테스트를 하기 위해서 파이프 라인을 생성 및 구성한다.

```
cat <<'EOF' | kubectl apply -f -
```

```
apiVersion: tekton.dev/v1
```

```
kind: Pipeline
```

```
metadata:
```

```
  name: go-pipeline
```

```
spec:
```

```
  workspaces:
```

```
    - name: shared-workspace
```

```
  tasks:
```

```
    - name: fetch-repo
```

```
      taskRef:
```

```
        name: git-clone
```

```
        bundle: qcr.io/tekton-releases/catalog/upstream/git-clone:0.9
```

계
속



빌드 파이프 라인 구성 3

```
params:
  - name: url
    value: "https://github.com/gookhyun/go-hello-world.git"
  - name: revision
    value: "main"
  - name: subdirectory
    value: ""
  - name: deleteExisting
    value: "true"
workspaces:
  - name: output
    workspace: shared-workspace
```

계
속



빌드 파이프 라인 구성 4

```
- name: build-go
  runAfter:
    - fetch-repo
  taskRef:
    name: go-build
  workspaces:
    - name: source
      workspace: shared-workspace
```

EOF



빌드 파이프 라인 실행 전

PVC가 구성이 안된 경우, 아래와 같이 구성 및 생성한다. 해당 자원이 없는 경우, 올바르게 실행이 안된다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: go-source-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

EOF



빌드 파이프 라인 실행

앞에서 구성한 파이프 라인에서 사용할 자원을 할당하여 실행한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  name: go-pipeline-run
spec:
  pipelineRef:
    name: go-pipeline
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: go-source-pvc
```

EOF



빌드 결과 화면

올바르게 구성이 되면, 다음과 같이 화면에 출력이 된다.

```
[build] go: downloading github.com/gin-gonic/gin v1.8.1  
[build] go: downloading ...  
[build] Compiled binary: hello-app
```



컨테이너 이미지 빌드

파이프라인

컨테이너 이미지 디스크 구성

소스코드를 컴파일 하였으면, 쿠버네티스에서 사용이 가능하도록 Containerfile로 패키징이 필요하다. 패키징 하기 위해서 다음과 같은 두 가지 작업이 필요하다.

1. Containerfile 혹은 Dockerfile
2. 이미지 빌드 후, 보관할 image-registry서버
3. 효율적인 빌드를 위한 buildah사용

위와 같은 조건으로 작업을 수행한다.



빌드 시 사용할 태스크

빌드 시, 다음과 같은 자원들을 사용하여 구성할 예정이다.

구성 요소	설명
git-clone	Git 저장소 복제 (Tekton Catalog)
build-image	Buildah or Kaniko 기반 이미지 빌드
push-image	레지스트리에 푸시
apply-deploy	kubectl apply로 쿠버네티스에 배포
PipelineRun	전체 실행 흐름
PVC	Git clone과 빌드 공유 목적
ServiceAccount	Registry 인증 및 쿠버네티스 권한 포함



쿠버네티스 클러스터 배포 파일 생성 1

쿠버네티스 클러스터에 배포 시 사용할 deployment, service YAML 파일을 생성한다.

```
cat <<EOF> go-hello-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-hello
spec:
  replicas: 1
  selector:
    matchLabels:
      app: go-hello
```



쿠버네티스 클러스터 배포 파일 생성 2

```
template:
  metadata:
    labels:
      app: go-hello
  spec:
    containers:
      - name: go-hello
        image: registry.dustbox.kr/gookhyun/go-app:latest
        ports:
          - containerPort: 8080
```

EOF



쿠버네티스 서비스 파일 생성

별로 만들지 말고 하나의 파일로 작성 및 구성한다.

```
cat <<EOF> go-hello-svc.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: go-hello-svc
```

```
spec:
```

```
  selector:
```

```
    app: go-hello
```

```
  ports:
```

```
    - port: 80
```

```
      targetPort: 8080
```

```
EOF
```



쿠버네티스 시크릿 및 서비스 계정 1

클러스터에서 사용할 SA계정 및 레지스트리 인증 계정 정보를 등록한다.

```
cat <<EOF> go-hello-svc.yaml
apiVersion: v1
kind: Secret
metadata:
  name: regcred
  annotations:
    tekton.dev/docker-0: https://registry.dustbox.kr
type: kubernetes.io/basic-auth
stringData:
  username: gookhyun
  password: xxxxxx
```

본인의 레지스트리 주소로 변경

계
속



쿠버네티스 시크릿 및 서비스 계정 1

클러스터에서 사용할 SA계정 및 레지스트리 인증 계정 정보를 등록한다.

```
cat <<EOF> go-hello-svc.yaml
apiVersion: v1
kind: Secret
metadata:
  name: regcred
  annotations:
    tekton.dev/docker-0: https://registry.dustbox.kr
type: kubernetes.io/basic-auth
stringData:
  username: gookhyun
  password: xxxxxx
```

본인의 레지스트리 주소로 변경

계
속



쿠버네티스 시크릿 및 서비스 계정 2

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pipeline-sa
secrets:
  - name: regcred
EOF
```



컨테이너 디스크 빌드 파일 구성

컨테이너 파일은 보통 다음과 같이 작성한다. 여기서는 Go언어 기반의 `helloworld` 프로그램 가지고 진행한다.

```
cat <<EOF> Containerfile
FROM golang:1.20 AS builder
WORKDIR /app
COPY . .
RUN go build -o hello-app

FROM debian:bullseye-slim
COPY --from=builder /app/hello-app /hello-app
ENTRYPOINT ["/hello-app"]
EOF
```



컨테이너 이미지 빌드 태스크 생성 1

컴파일 및 디스크 이미지 디스크 빌드를 작업으로 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: buildah-go-image
spec:
  params:
    - name: IMAGE
      description: Target image name
    - name: CONTEXT
      description: Build context path
      default: .
```



컨테이너 이미지 빌드 태스크 생성 2

```
workspaces:  
  - name: source  
steps:  
  - name: build-push  
    image: quay.io/buildah/stable:v1.27.0  
    securityContext:  
      privileged: true  
    workingDir: $(workspaces.source.path)/$(params.CONTEXT)  
    script: |  
      buildah bud -f Containerfile -t $(params.IMAGE) .  
      buildah push $(params.IMAGE)
```

EOF



애플리케이션 배포 시도(실패) 1

이번에는 이미지를 쿠버네티스에 배포하는 작업을 구성한다. 여기서는 SA 및 RBAC구성하지 않았기에 올바르게 동작하지 않는다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: Tekton.dev/v1
kind: Task
metadata:
  name: apply-deploy
spec:
  params:
    - name: YAML_PATH
      default: k8s/deployment.yaml
```

계속



애플리케이션 배포 시도(실패) 2

```
workspaces:  
  - name: source  
steps:  
  - name: apply  
    image: bitnami/kubectl:latest  
    script: |  
      kubectl apply -f $(workspaces.source.path)/$(params.YAML_PATH)
```

EOF

배포 시 필요한 YAML파일은 별도로 구성해야 한다.
여기서는 임의로 가져와서 사용한다.



빌드 파이프 라인 구성 1

모든 태스크를 동시에 수행하기 위해서 파이프 라인을 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: go-app-cicd
spec:
  params:
    - name: IMAGE
  workspaces:
    - name: shared-workspace
```



빌드 파이프 라인 구성 2

```
tasks:  
  - name: fetch-repo  
    taskRef:  
      name: git-clone  
      bundle: gcr.io/tekton-releases/catalog/upstream/git-clone:0.9  
    params:  
      - name: url  
        value: "https://github.com/gookhyun/go-hello-world.git"  
      - name: revision  
        value: "main"
```

이전에 만든 작업 결과물이 공유가 된다.

계
속



빌드 파이프 라인 구성 3

```
workspaces:  
  - name: output  
    workspace: shared-workspace
```



빌드 파이프 라인 구성 4

```
- name: build-push
  runAfter: [fetch-repo]
  taskRef:
    name: buildah-go-image
  params:
    - name: IMAGE
      value: "${params.IMAGE}"
    - name: CONTEXT
      value: "."
  workspaces:
    - name: source
      workspace: shared-workspace
```



빌드 파이프 라인 구성 5

```
- name: deploy
  runAfter: [build-push]
  taskRef:
    name: apply-deploy
  params:
    - name: YAML_PATH
      value: "k8s/deployment.yaml"
  workspaces:
    - name: source
      workspace: shared-workspace
```

EOF



빌드 파이프라인 실행 1

앞에서 구성한 파이프 라인을 실행한다. 여기에 빌드 시 필요한 파라미터를 추가한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: Tekton.dev/v1
kind: PipelineRun
metadata:
  name: go-app-cicd-run
spec:
  pipelineRef:
    name: go-app-cicd
  params:
    - name: IMAGE
      value: registry.dustbox.kr/tangt64/go-app:latest
```

계
속



빌드 파이프라인 실행 2

앞에 내용 계속 이어서...

```
workspaces:  
  - name: shared-workspace  
    persistentVolumeClaim:  
      claimName: go-source-pvc  
    serviceAccountName: pipeline-sa  
EOF
```



실행 순서(runafter)

파이프라인

PIPELINE ORDERING

파이프라인은 자유롭게 **작업 순서 변경(ordering)**이 가능하다. 이를 통해서 각 작업마다 원하는 방식으로 작업 순서를 지정할 수 있다. 이미 위에서 기능 학습을 하였지만, **작업(task) 및 단계(step)**은 위치에 따라서 실행하는 순서가 달라진다.

그래서 태스크에서 구성한 순서가 아닌, 사용자가 원하는 순서대로 진행을 하고 싶은 경우, YAML코드에 사용자가 원하는 작업 이름을 순서대로 명시하면 된다.

아래 슬라이드의 내용처럼, 간단하게 여러 개의 작업을 구성하여 파이프라인을 통해서 실행 순서를 재구성 한다.



PIPELINE ORDERING - TASK 1

다음과 같이 작업에 대해서 순서 선언이 가능하다. 먼저, 실험용 작업을 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: pipeline-ordering-task
spec:
  params:
    - name: task-name
      type: string
    - name: time
      type: string
      default: ""
```



PIPELINE ORDERING - TASK 2

steps:

- name: first-task
image: quay.io/centos/centos:stream9
command:
 - /bin/bashargs: ['-c', 'echo Ran to the first task \${params.task-name}']
- name: second-task
image: quay.io/centos/centos:stream9
command:
 - /bin/bashargs: ['-c', 'echo Ran to the second task \${params.time}']



PIPELINE ORDERING - TASK 3

```
- name: logger
  image: quay.io/centos/centos:stream9
  command:
    - /bin/bash
  args: ['-c', 'echo Ran to the last task $(date +%d/%m/%Y %T) - Task
$(params.task-name) Completed']
EOF
```



PIPELINE 생성 및 순서 1

앞에서 작성한 작업들을 파이 라인으로 순서대로 실행하도록 작성한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1beta1  
kind: Pipeline  
metadata:  
  name: pipeline-ordering-pipeline
```



PIPELINE 생성 및 순서 2

```
spec:  
  tasks:  
    - name: first  
      params:  
        - name: task-name  
          value: A  
        - name: time  
          value: "2PM"  
  taskRef:  
    name: first-task
```



PIPELINE 생성 및 순서 3

- **name: second**

params:

- name: task-name

value: "This is the Task 2"

taskRef:

name: first-task

- **name: third**

params:

- name: task-name

value: "This is the Task 3"

- name: time

value: "3PM"

계
속



PIPELINE 생성 및 순서 4

```
- name: fourth
  params:
    - name: task-name
      value: "This is the Task 4"
  taskRef:
    name: logger
```

EOF



PIPELINE ORDERING - 확인

적용 및 반영한 내용을 확인한다. 순서대로 실행이 되는지 확인한다.

```
tkn pipeline start pipeline-ordering-pipeline --showlog
```



PIPELINE ORDERING 지시자

앞에서 사용하였던 **ordering** 작업 순서를 반영한다. 먼저 아래와 같은 지시자를 통해서 작업 순서를 반영한다.

- **runAfter**
- **when**
- **finally**

일반적 많이 사용하는 지시자는 **runafter**.

- **name: test**
 - runAfter:**
 - **build**



PIPELINE 실행 순서 반영 및 수정 1

```
kubectl edit pipeline pipeline-ordering-pipeline
spec:
  tasks:
    - name: first
      params:
        - name: task-name
          value: A
        - name: time
          value: "2PM"
      taskRef:
        name: first-task
```



PIPELINE 실행 순서 반영 및 수정 2

```
- name: second
  params:
    - name: task-name
      value: "This is the Task 2"
  taskRef:
    name: first-task
  runAfter:
    - first
```



PIPELINE 실행 순서 반영 및 수정 3

- name: third

params:

- name: task-name
value: "This is the Task 3"
- name: time
value: "3PM"

runAfter:

- second



PIPELINE 실행 순서 반영 및 수정 4

```
- name: fourth
  params:
    - name: task-name
      value: "This is the Task 4"
  taskRef:
    name: logger
  runAfter:
    - third
```



파이프라인 연습문제

연습문제

연습문제

다음과 같이 파이프라인 및 태스크를 구성해서 작업을 수행한다.

1. 모든 작업은 quay.io/centos/centos:stream9 이미지를 사용한다.
2. 첫번째 작업은 사용자에게 메시지를 입력 받는다.
 1. uname이름으로 사용자 이름을 입력 받는다.
 2. 입력 받은 이름은 반드시 출력해야 한다.
 3. 사용자 이름을 uname.txt파일로 저장한다.
3. 두 번째 작업은 사용자의 이메일 주소를 입력 받는다.
 1. uemail이름으로 사용자 메일 주소를 입력 받는다.
 2. 입력 받은 이름은 반드시 출력해야 한다.
 3. 사용자 메일 주소를 uemail.txt파일로 저장한다.
4. 위의 모든 작업을 파이프라인으로 생성 후 동시에 작업을 수행하도록 한다.

워크스페이스

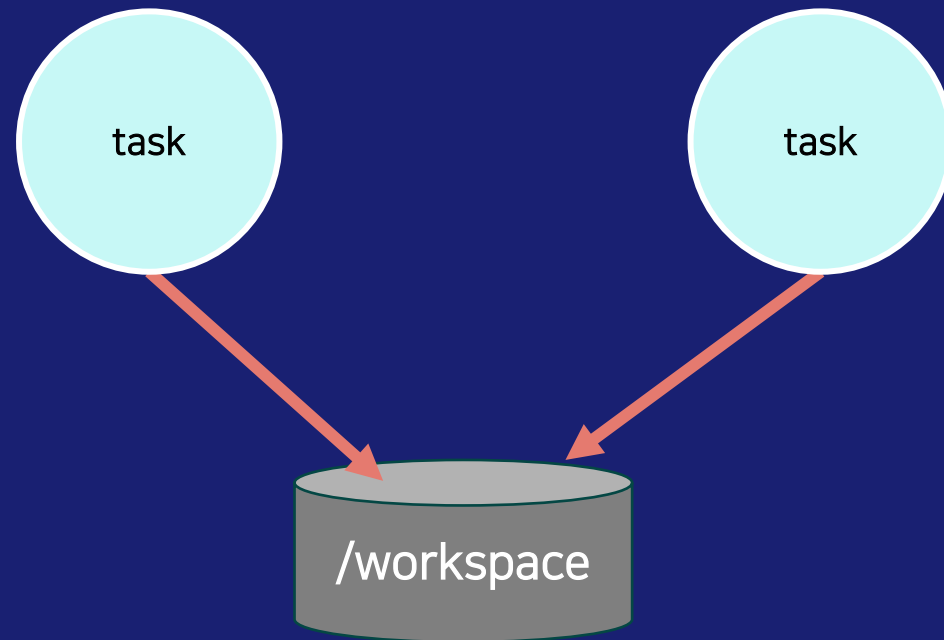
데이터 공유

워크스페이스

소개

워크스페이스 설명

워크 스페이스는 작업에서 생성 된 데이터를 저장한다. 저장은 POD의 **특정 공유 볼륨** 혹은 **PVC**를 통해서 컨테이너 데이터를 저장한다.



워크스페이스 설명

워크스페이스와 비슷한 **결과(results)** 경우에는 각 단계에서 발생한 결과를 저장할 수 있지만, 여기에는 최대 크기가 4k(4096bytes)에 대한 제한이 있다.

<https://github.com/tektoncd/pipeline/issues/4060>

상태	이유	설명
False	TaskRunResultTooLarge	"TaskRun"에서 크기가 4096바이트를 넘어가는 경우, 이 옵션을 통해서 허용한다.



지원하는 자원 형태

워크스페이스에서 지원하는 저장소 형태는 다음과 같다.

- `emptyDir`
- `ConfigMap`
- `Secret`

쿠버네티스에서 제공하는 볼륨 `Persistent Volume`, `Persistent Volume Claim`를 지원한다.

`PV/PVC`를 사용한 경우, 앞에서 학습하였던 `finally`를 사용하여 종료 시, 내부 데이터를 제거해야 한다. 그렇지 않는 경우, 기존 데이터가 남아 있기 때문에, 다른 컨테이너 사용자에게 기존 내용이 노출이 될 수도 있다.

- `PersistentVolume`
- `PersistentVolumeClaim`



선언 방식

워크스페이스를 선언하기 위해서 다음과 같이 YAML파일에 작성한다.

- **workspaces:** 부분은 워크스페이스에서 생성한 이름을 보통 적는다.
- **workingDir:** 명령어가 수행할 작업 위치를 명시하며, 보통 테크톤 변수를 통해서 선언이 가능하다.

명시한 이름은 `/workspaces/<WORKSPACE_DIR_NAME>/`으로 생성이 된다. 예를 들어서 위에서 작성한 `source`이름으로 작성한 경우 `/workspace/source`으로 디렉터리 생성이 된다.

```
workspaces:  
  - name: source  
steps:  
  - name: clone  
    image: quay.io/centos/centos:stream9  
    workingDir: $(workspaces.source.path)
```



저장소 형식

워크스페이스

저장소 형식

앞에서 이야기 하였지만, 워크스페이스에서는 쿠버네티스에서 사용하는 저장소 유형을 전부 사용이 가능하다. 하지만, 데이터가 많이 발생하는 경우 일반적으로 다음과 같이 사용한다.

1. `emptyDir`

2. `PV/PVC`

3. `StorageClass`(권장)

가급적이면 `스토리지 클래스(StorageClass)`으로 구성한다. 기본적으로 `NFS CSI(Container Storage Interface)`가 구성이 안되어 있다는 조건으로 `emptyDir:`를 사용한다.



저장소 사용 예 1

앞에서 사용했던 예제 기반으로 설명하면 다음과 같다.

```
workspaces:  
  - name: source  
steps:  
  - name: clone  
    image: quay.io/centos/centos:stream9  
    workingDir: $(workspaces.source.path)
```



저장소 사용 예 2

별도로 명시되지 않는 경우, `emptyDir`으로 구성이 되며, 만약 PVC를 통해서 저장소를 전달하는 경우 다음처럼 작성한다. 미리 구성한 PVC의 이름을 작성하면 된다.

```
workspaces:  
  - name: output  
    persistentVolumeClaim:  
      claimName: tekton-tutorial-sources
```



첫 워크스페이스 생성

워크스페이스

간단한 워크스페이스 생성

예를 들어서 깃헙에서 특정 소스코드를 복제 후, 실행하는 작업이 필요한 경우, 이때 워크스페이스가 필요하다. 간단하게 만든 워크스페이스는 다음과 같은 작업을 수행한다.

1. 깃헙에서 소스코드, `duststack-osp-auto`를 복제한다.
2. 앤서블이 실행이 가능하도록 패키지를 설치한다.(RPM, PIP)
3. 복제한 코드를 앤서블 명령어로 간단하게 코드 검증 및 확인한다.



간단한 워크스페이스 예제

저장소는 간단하게 emptyDir를 사용하며, 이를 통해서 소스코드를 저장 후 앤서블을 실행한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-workspace-clone-github
spec:
  params:
    - name: repo
      type: string
      description: Git Repository clone from github
      default: https://github.com/tangt64/duststack-osp-auto
  workspaces:
    - name: source
```



간단한 워크스페이스 예제

앞에 내용 계속 이어서...

steps:

- name: clone

image: quay.io/centos/centos:stream9

workingDir: \$(workspaces.source.path)

script: |

dnf install git -y

git clone -v \$(params.repo) \$(workspaces.source.path)

- name: list

image: quay.io/centos/centos:stream9

workingDir: \$(workspaces.source.path)

script: |

ls -l \$(workspaces.source.path)

계
속



간단한 워크스페이스 예제

- **name: ansible**

image: quay.io/centos/centos:stream9

workingDir: \$(workspaces.source.path)/duststack-osp-auto

script: |

dnf install ansible -y

ansible-playbook --syntax-check playbooks/osp-install.yaml



실행

다음 명령어로 실행한다.

```
tkn task start demo-clone-GitHub --showlog
? Value for param `repo` of type `string`
? Please give specifications for the workspace: source
? Name for the workspace : source
? Value of the Sub Path :
? Type of the Workspace : emptyDir
? Type of EmptyDir :
```



워크스페이스에서 작업 실행

워크스페이스

TaskRun 설명

태스크 실행(TaskRun)은 특정 작업을 실행 시 사용한다.

예를 들어서 앞에서 만든 작업, `demo-clone-github`를 `tr(TaskRun)`를 통해서 실행한다. 이전에는 작업을 실행 시 `tkn task start <TASK_ID>`를 통해서 실행하였다. 태스크 실행을 사용하는 경우, 코드 기반으로 작업 수행이 가능하다.

```
tkn task start demo-clone-GitHub --showlog
```



TaskRun 적용

다음처럼 변경 및 실행이 가능하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: TaskRun
metadata:
  generateName: demo-tr-clone-github
spec:
  workspaces:
    - name: source
      emptyDir: {}
  taskRef:
    name: demo-workspace-clone-github
EOF
```



TaskRun 확인

아래 명령어로 실행한다.

```
tkn taskrun list  
tkn taskrun start demo-tr-clone-github  
tkn taskrun logs demo-tr-clone-github
```



파이프라인 추가

워크스페이스

파이프라인에 워크스페이스 설명

파이프라인에 워크스페이스를 추가해서 사용하면, 손쉽게 반복적으로 사용이 가능하다.

기존에 사용하였던 깃헙 코드를 조금 더 단순하게 만들어서 테스트를 한다. 첫 번째는 **파이프라인**, 나머지 두개는 **태스크** 내용이 작성이 된다.



파이프 라인에서 사용할 태스크 생성 1

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1beta1  
kind: Task  
metadata:  
  name: demo-workspace-clone-github-pipeline  
spec:
```



파이프 라인에서 사용할 태스크 생성 2

```
params:  
  - name: repo  
    type: string  
    description: Git Repository clone from github  
    default: https://github.com/tangt64/duststack-osp-auto
```

workspaces:

- name: source

steps:

- name: clone
 image: quay.io/centos/centos:stream9
 workingDir: **\$(workspaces.source.path)**
 script: |
 dnf install git -y
 git clone -v \$(params.repo)

EOF



파이프 라인에서 사용할 태스크 생성 3

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-workspace-list-github-pipeline
spec:
  workspaces:
    - name: gitcode
```



파이프 라인에서 사용할 태스크 생성 4

앞에 내용 계속 이어서...

```
steps:
```

```
  - name: list
```

```
    image: quay.io/centos/centos:stream9
```

```
    workingDir: ${workspaces.source.path}
```

```
    script: |
```

```
      ls -l
```

```
EOF
```



파이프라인 및 워크스페이스 구성 1

위에서 작성한 태스크를 파이프 라인으로 생성 및 구성한다.

```
cat <<'EOF' | kubectl apply -f -  
---  
apiVersion: tekton.dev/v1beta1  
kind: Pipeline  
metadata:  
  name: demo-clone-list-from-pipeline  
spec:  
  workspaces:  
    - name: gitcode
```



파이프라인 및 워크스페이스 구성 2

```
tasks:  
  - name: clone  
    taskRef:  
      name: demo-workspace-clone-github-pipeline  
workspaces:  
  - name: source  
    workspace: gitcode
```



파이프라인 및 워크스페이스 구성 3

앞에 내용 계속 이어서...

```
- name: list
  taskRef:
    name: demo-workspace-list-github-pipeline
workspaces:
  - name: source
    workspace: gitcode
runAfter: ['clone']
  - clone
```



워크스페이스

파이프라인에 영구적 데이터 저장소 구성

PVC기반 워크스페이스 설명

PVC기반으로 구성하기 위해서는 먼저 쿠버네티스 클러스터에 PV/PVC가 구성이 되어 있어야 한다. NFS서버가 구성이 안되어 있으면 아래와 같이, NFS서버 및 CSI-NFS드라이버를 설치한다.

1. <https://github.com/kubernetes-csi/csi-driver-nfs>
2. <https://github.com/kubernetes-csi/csi-driver-nfs/blob/master/docs/install-csi-driver-v4.7.0.md>

```
curl -skSL https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.7.0/deploy/install-driver.sh | bash -s v4.7.0 --  
kubectl -n kube-system get pod -o wide -l app=csi-nfs-controller  
kubectl -n kube-system get pod -o wide -l app=csi-nfs-node
```



PVC기반 워크스페이스 설명

오픈스택 랩 사용자 경우에는 스크립트에서 같이 설치가 되기 때문에 별도의 과정은 필요 없다.



NFS서버 구성

컨트롤 노드에서 NFS서버를 구성한다. `showmount` 명령어로 올바르게 조회가 되는지 확인한다.

```
dnf install nfs-utils
cat <<EOF> /etc/exports
/workspaces/ *(rw)
EOF
mkdir -m 777 -p /workspaces
systemctl enable --now nfs-server
export -avrs
showmount -e master.example.com
```



스토리지 클래스 및 PVC 확인

올바르게 구성이 되었는지 확인한다. 이 랩은 이 부분까지 자동으로 구성이 된다.

```
kubectl apply -f storageclass-configure.yaml
```

```
kubectl apply -f tekton-pvc.yaml
```

```
kubectl get sc
```

nfs-csi (default)	nfs.csi.k8s.io	Delete	Immediate
-------------------	----------------	--------	-----------

```
kubectl get pvc
```

tekton-pvc	Bound	pvc-fc50e03e-	1Gi	RWX	nfs-csi	13s
------------	-------	---------------	-----	-----	---------	-----

무조건 한 개 이상의 SC가 필요하다!



테크톤 PVC 연결

워크스페이스에 사용하는 저장소를 PVC로 연결해서 사용한다. 이번에 사용하는 PVC는 이름을 명시한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: PipelineRun
metadata:
  generateName: clone-and-list-pr-
spec:
  pipelineRef:
    name: demo-workspace-clone-list-github-pipeline
  workspaces:
    - name: gitcode
      persistentVolumeClaim:
        claimName: tekton-pvc
```

name: 지시자를 사용하여도 상관은 없다. 다만, 이름이 고정으로 되기 때문에, 두 번 실행은 되지 않는다.

계
속



테크톤 PVC 연결

앞에 내용 계속 이어서...

```
tkn pr list
```



파이프라인과 PVC 연결 확인 1

올바르게 PVC가 구성이 되는지 확인한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: PipelineRun
metadata:
  generateName: clone-and-list-pr-
spec:
  pipelineSpec:
    workspaces:
      - name: gitcode
```



파이프라인과 PVC 연결 확인 2

```
tasks:  
  - name: clone  
    taskRef:  
      name: demo-workspace-clone-github-pipeline  
workspaces:  
  - name: source  
    workspace: gitcode
```



파이프라인과 PVC 연결 확인 3

```
- name: list
  taskRef:
    name: demo-workspace-list-github-pipeline
  workspaces:
    - name: source
      workspace: gitcode
  runAfter:
    - clone
```

EOF



파이프라인과 STATIC PVC 연결

고정이 아닌, 필요할 때 마다 PVC를 생성해서 워크스페이스를 구성하는 경우 아래와 같이 구성한다.

```
kubectl edit clone-and-list-pr-  
workspaces:  
- name: gitcode  
  volumeClaimTemplate:  
    spec:  
      accessModes:  
      - ReadWriteOnce  
      resources:  
        requests:  
          storage: 1Gi  
tkn pipelinerun list
```



연습문제

워크스페이스

연습문제

다음과 같이 파이프라인 및 태스크를 구성해서 작업을 수행한다.

1. 모든 작업은 quay.io/centos/centos:stream9 이미지를 사용한다.
2. 레지스트리 서버를 간단하게 생성 및 구성한다.
 1. 마스터 노드에 docker-register서버를 구성한다.
 2. 포트번호는 5000번을 사용해서 동작하도록 한다.
 3. 구성은 Podman기반으로 한다.
3. 첫 번째 태스크는 다음과 같은 작업을 수행한다.
 1. 사용자가 명시한 주소의 소스코드를 내려받기 한다.
 2. 소스코드는 buildeah를 통해서 이미지 빌드를 수행한다.
 3. 해당 이미지는 이미지 빌드가 완료가 되면 내부 레지스트리 서버에 저장한다.
4. 세 번째 태스크는 다음과 같은 작업을 수행한다.
 1. 구성된 이미지 기반으로 웹 서비스를 구성한다.
 2. kubectl명령어로 빌드한 이미지 기반으로 웹 서비스를 구성한다.
 3. 해당 웹 서비스는 노드포트 38080으로 접근이 가능해야 한다.
5. 모든 작업들은 파이프라인으로 수행이 되어야 한다.

디버깅

디버깅

파이프라인 및 태스크 정리

디버깅 명령어 방법

테크톤에서 디버깅 방식은 두가지가 있다.

1. **kubectl**명령어를 통한 디버깅

2. **tkn**명령어를 통한 디버깅

tkn명령어 기반으로 하는 경우, 어떠한 **태스크/파라미터**를 찾지 못하는지 메시지를 통해서 확인이 가능하지만, 이 미지나 혹은 컨테이너에서 발생한 오류 메시지는 확인이 불가능하다.

그래서 **kubectl**명령어를 통해서 좀 더 자세하게 확인을 해야 한다.

kubectl get tasks	→ kubectl get pods
kubectl get pipeline	→ kubectl get pods
kubectl describe pod <TASK_NAME>	→ tkn task describe <TASK_NAME>
kubectl logs tasks <TASK_NAME>	→ kubectl log pod/<POD_NAME>



DEBUG-TASK 구성 및 생성 1

태스크 디버깅 위해서 다음과 같이 작성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: exit-debug-task
spec:
  params:
    - name: text
      type: string
    - name: exitcode
      type: string
```



DEBUG-TASK 구성 및 생성 2

steps:

- name: log
image: quay.io/centos/centos:stream9
command:
 - /bin/bashargs: ["-c", " echo \$(params.text)"]
- name: exit
image: quay.io/centos/centos:stream9
command:
 - /bin/bashargs: ["-c", "echo 'Exiting with code \$(params.exitcode)' && exit \$(params.exitcode)"]

EOF



DEBUG-TASK 로그 1

올바르게 실행이 되었는지, 디버깅 작업을 구성하여 결과물을 확인한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: log-and-exit
spec:
  params:
    - name: text
      type: string
    - name: exitcode
      type: string
```



DEBUG-TASK 로그 1

앞에 내용 계속 이어서...

```
steps:
```

```
- name: log
```

```
  image: quay.io/centos/centos:stream9
```

```
  command:
```

```
    - /bin/bash
```

```
  args: ['-c, 'echo ${params.text}']
```

```
EOF
```



파이프 라인을 통한 실행 및 확인 1

위의 작업을 파이프라인을 통해서 호출 및 실행한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1beta1  
kind: Pipeline  
metadata:  
  name: exit-debug-pipeline
```



파이프 라인을 통한 실행 및 확인 2

앞에 내용 계속 이어서...

```
spec:
  tasks:
  - name: clone
    taskRef:
      name: log-and-exit
    params:
      - name: text
        value: "Simulating git clone"
      - name: exitcode
        value: "0"
```

EOF



DEBUG-PIPELINE

위의 작업들을 파이프라인을 통해서 작업을 구성한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: tekton.dev/v1beta1  
kind: Pipeline  
metadata:  
  name: exit-debug-pipeline
```



DEBUG-PIPELINE

```
spec:
  tasks:
  - name: clone
    taskRef:
      name: log-and-exit
    params:
      - name: text
        value: "Simulating git clone"
      - name: exitcode
        value: "0"
```



DEBUG-PIPELINE

```
- name: unit-tests
  taskRef:
    name: exit-debug-task
  params:
    - name: text
      value: "Simulating unit testing"
    - name: exitcode
      value: "0"
  runAfter:
    - clone
```



DEBUG-PIPELINE

```
- name: deploy
  taskRef:
    name: exit-debug-task
  params:
    - name: text
      value: "Simulating deployment"
    - name: exitcode
      value: "0"
  runAfter:
    - unit-tests
```



DEBUG-TASK 실행 및 결과 확인

올바르게 구성이 되었는지 확인한다.

```
tkn pipeline start exit-debug-pipeline --showlog
```



DEBUG-TASK VALUE

올바르게 동작하는지 확인하기 위해서 `exit-debug-task.yaml` 파일에 아래와 같이 수정한다. 기존 값 "0"을 "1"으로 변경한다.

```
- name: unit-tests
  taskRef:
    name: exit-debug-task
  params:
    - name: text
      value: "Simulating unit testing"
    - name: exitcode
      value: "0" → "1"
  runAfter:
    - clone
```



DEBUG-TASK RUN 2

다음 명령어로 등록 및 실행한다.

```
kubectl -f exit-debug-pipeline.yaml  
tkn pipeline start exit-debug-task --showlog
```



FINALLY/CLEANUP

Finally, Cleanup작업은 특정 작업이 완료가 된 후, 특정 작업을 내용을 제거 시 사용한다. 보통 이 부분은 **PV/PVC**를 사용할 때 이 명령어를 사용하여 처리한다.

아래 예제들은 간단하게 메시지 출력 후, PVC의 내용을 **finally**명령어를 통해서 작업 내용을 제거한다.



CLEANUP

작업 마무리 시, 다음과 같이 작업을 수행한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: task-cleanup
spec:
  steps:
  - name: clean
    image: quay.io/centos/centos:stream9
    command:
      - /bin/bash
    args: ['-c', 'echo Cleaning up!']
EOF
```



FINALLY

위의 내용에 다음과 같은 내용을 추가하여 기능을 확장한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: exit-debug-pipeline
spec:
  tasks:
...
  finally:
  - name: cleanup-task
    taskRef:
      name: cleanup
```



확인

다음과 같이 적용 및 확인한다.

```
tkn pipeline start exit-debug-pipeline --showlog
```



디버깅

연습문제

연습문제

아래 작업을 등록하여 올바르게 실행이 되는지 확인한다.

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: echo-task
spec:
  params:
    - name: message
      type: string
  steps:
    - name: echo
      image: alpine
      script: |
        echo "${params.msg}"
```

연습문제

아래 코드를 올바르게 동작하도록 수정. 파일 이름은 적절하게 생성 후, 파이프라인으로 실행이 되어야 함.

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: generate-result
spec:
  steps:
    - name: echo
      image: alpine
      script: |
        echo "myvalue" > $(results.myoutput.path)
```

연습문제

앞에 내용 계속 이어서... 위의 파이프라인에서 사용하는 작업파일.

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: debug-pipeline-2
spec:
  tasks:
    - name: generate
      taskRef:
        name: generate-result
```

연습문제

앞에 내용 계속 이어서... 위의 파이프라인에서 사용하는 작업파일.

```
- name: use-result
  taskRef:
    name: echo-task
  params:
    - name: message
      value: "${tasks.generate.results.myoutput}"
```

연습문제

해당 파이프라인에 어떠한 문제가 있는지 확인 후 코드를 적절하게 수정. 앞에 echo-task와 연결됨.

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: debug-pipeline-2
spec:
  tasks:
    - name: generate
      taskRef:
        name: generate-result
```

연습문제

해당 파이프라인에 어떠한 문제가 있는지 확인 후 코드를 적절하게 수정. 앞에 echo-task와 연결됨.

```
- name: use-result
  taskRef:
    name: echo-task
  params:
    - name: message
      value: "${tasks.generate.results.myoutput}"
```


테크톤 조건식

조건식 활용 및 사용하기

조건식

소개

소개

테크톤에서 조건식은 앤서블과 비슷하게 제공하지만, 기본적으로 Go Lang에서 사용하는 템플릿 형식을 사용한다.

1. <https://go.dev/ref/spec>
2. https://tekton.dev/docs/triggers/cel_expressions/
3. <https://tekton.dev/docs/pipelines/pipelines/>

입력 혹은 출력에 결과 내용을 다음과 같이 조건으로 사용이 가능하다.

when:

- input: "true"
operator: IN
values: ["true"]



조건 지시자 소개

기본적으로 많이 사용하는 조건식 명령어는 아래와 같다.

항목	설명
input	비교할 값 (일반적으로 파라미터 또는 결과 값 사용)
operator	비교 연산자 (in, notin)
values	비교 대상 값 배열
사용 위치	Pipeline 또는 Task 내 when 필드 사용
동작 방식	조건이 만족할 때만 해당 Task 실행



조건 지시자 설명 1

좀 더 자세한 사용 방법은 아래 표를 참고한다.

구성 요소	설명	예시 구문
input	조건식에 입력되는 값. 생략 시 빈 문자열이 기본값으로 사용됨.	- 고정값 예: "ubuntu" - 변수 예: \$(params.image) - Task 결과 예: \$(tasks.task1.results.image) - 배열 값 예: \$(tasks.task1.results.array-results[1])
operator	입력 값과 values 배열의 관계를 나타내는 연산자. 유효한 연산자를 반드시 지정해야 함.	- in - notin



조건 지시자 설명 2

좀 더 자세한 사용 방법은 아래 표를 참고한다.

구성 요소	설명	예시 구문
values	비교 대상이 되는 문자열 배열. 반드시 비어 있지 않아야 함.	- 배열 파라미터 예: ["\$(params.images[*])"] - Task 결과 배열 예: ["\$(tasks.task1.results.array-results[*])"] - 고정값 포함 예: "ubuntu" - 변수 포함 예: ["\$(params.image)", ["\$(tasks.task1.results.image)"]]



지시자 코드 예제 1

입력 사용 시, 보통 아래와 같이 코드로 표현한다.

```
- name: conditional-task
  taskRef:
    name: echo-hello
  when:
    - input: "${params.environment}"
      operator: in
      values: ["prod", "staging"]
```



지시자 코드 예제 2

아래 코드를 보면 알겠지만, 앤서블에서 사용하는 조건식 when과 거의 비슷하게 사용한다. 변수 number에 값이 10이 들어오면, **winmsg**작업을 실행한다. 조건식은 가급적이면 대문자로 작성한다.

테크톤에서 조건식을 적용하면 다음과 같이 사용이 가능하다.

```
- name: win
  params:
    - name: text
      value: winner
  taskRef:
    name: winmsg
  when:
    - input: $(params.number)
      operator: in      # IN
      values: ["10"]
```



조건식/파라미터 사용하기

조건식

태스크 생성 1

조건식 테스트를 위해서 작업을 아래와 같이 작성한다. 여기서는 시간에 관련된 서비스를 간단하게 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-expression-task-time-show-timedate
spec:
```



태스크 생성 2

```
params:
  - name: locale
    type: string
    default: America/New_York
steps:
  - name: show-timedate
    image: quay.io/centos/centos:stream9
    script: |
      export TZ=$(params.locale)
      DATE=$(date +%d/%m/%Y\ %T)
      echo [$DATE] - $(params.locale)
```

EOF



파이프 라인 적용 1

아래와 같이 파이프 라인을 생성한다. 앞에서 생성한 작업에 변수를 전달한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: demo-expression-pipeline-time-show-timedate
spec:
  params:
    - name: timezone
      description: set to TZ
      type: string
```



파이프 라인 적용 1

```
tasks:  
  - name: timezone  
    params:  
      - name: timezone  
        value: Asia/Seoul  
    taskRef:  
      name: demo-expression-task-time-show-timedate  
when:  
  - input: ${params.timezone}  
    operator: in  
    values: ["Asia/Seoul"]
```

when를 사용하여 조건식 적용

계
속



결과 확인하기

아래 명령어로 실행한다.

이를 통해서 조건식 검사가 올바르게 되는지 확인한다. 여기서는 기본값이 **Asia/Seoul**으로 되어 있기 때문에 문제 없이 실행이 되어야 한다.

```
tkn pipeline list  
tkn task list
```



NOTIN 설명

기존에 사용하던 파일에 다음과 같이 수정한다. NOTIN은 말 그대로 조건에 포함이 되지 않는 경우 참으로 실행이 된다.

두 개 지시자를 비교하면 다음과 같다.

구분	IN	NOTIN
의미	값이 목록에 포함되면 실행	값이 목록에 포함되지 않으면 실행
성격	허용 목록 (Allowlist)	차단 목록 (Denylist)
사용 목적	특정 값만 허용	특정 값만 제외
기본 감각	이 중 하나면 OK	이건 빼고 다 OK



IN/NOTIN 코드 구성

아래와 같이 NOTIN를 적용한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: demo-expression-pipeline-time-show-timedate
spec:
  params:
    - name: timezone
      description: set to TZ
      type: string
```



비교용 IN 구성

먼저 IN 코드를 적용한다. 이건, 앞에서 사용한 코드이다.

```
tasks:
  - name: timezone
    params:
      - name: timezone
        value: Asia/Seoul
    taskRef:
      name: demo-expression-task-time-show-timedate
  when:
    - input: $(params.timezone)
      operator: IN
      values: ["Asia/Seoul"]
```

이 부분을 아래처럼 변경한다.

계
속



비교용 NOTIN

위의 내용의 마지막에 아래 내용을 추가한다.

```
- name: timezone-default
  params:
    - name: timezone
      value: Asia/Tokyo
  taskRef:
    name: demo-expression-task-time-show-timedate
  when:
    - input: $(params.timezone)
      operator: NOTIN ← 이 부분 변경
      values: ["Asia/Tokyo"]
```

EOF



적용 및 확인

아래 명령어로 실행한다. 어떠한 결과가 나오는지 확인한다.

```
tkn pipeline list
```

```
tkn pipeline start demo-expression-pipeline-show-timedate-notin
```



깃 서버 인증

HTTP 혹은 SSH 인증

인증 구성

설명 및 구성

인증?!

테크톤은 기본적으로 GIT 서버에 로그인 기능을 제공하지 않는다. 그러한 이유로 쿠버네티스에서 제공하는 방법으로 접근이 가능하다.

기본적으로 접근 방법 시 두 가지 쿠버네티스 자원을 사용한다.

1. ConfigMap
2. Secret

이 두 가지를 테크톤에서 활용하여 Git 및 Registry 서버에 접근이 가능하다.



GIT 서버 로그인 1

내부 깃 서버에서 소스코드를 다운로드 받기 위해서 아래와 같이 코드를 작성 후, 저장소에 접근한다.
여기서는 GIT 컨테이너 이미지를 통해서 소스코드를 내려 받는다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: demo-github-list
spec:
  params:
    - name: private-repo
      type: string
```



GIT 서버 로그인 2

steps:

```
- name: clone
  image: alpine/git
  script: |
    mkdir /temp && cd /temp
    git clone $(params.private-repo) .
    ls -lR .
```

알파인 이미지로 서버에
접근

EOF



GIT 서버 토큰 로그인 1

깃 서버에서는 다음과 두 가지 방식으로 보통 인증을 제공한다.

1. 토큰
2. 사용자 및 비밀번호

사용자 및 비밀번호를 통해서 접근 시, 다음과 같이 접근이 가능하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: git-auth
  annotations:
    tekton.dev/git-0: https://github.com
```



GIT 서버 토큰 로그인 2

```
type: kubernetes.io/basic-auth
stringData:
  username: gituser
  password: gitpassword
EOF
```



GIT 서버 SSH 로그인

SSH 기반으로 접근이 필요한 경우 아래와 같이 작성 후 접근이 가능하다.

접근 시, `known_hosts` 문제를 막기 위해서 미리 `fingerprint`를 넣어준다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: git-auth
  annotations:
    tekton.dev/git-0: github.com
type: kubernetes.io/ssh-auth
```



GIT SERVER SSH

stringData:

ssh-privatekey: |

——BEGIN OPENSSH PRIVATE KEY——

——END OPENSSH PRIVATE KEY——

known_hosts: github.com,140.82.112.4 ssh-rsa

AAAAB3NzaC1yc2EAAAABIwAAAQEAq2A7hRGmdnm9tUDb09IDSwBK6TbQa+PXYPcPy6rbTrTtw7P
HkccKrpp0yVhp5HdEIcKr6pLlVDBf0LX9QUsyCOV0wzfjIJNlGEYsdLLJizHhbn2mUjvSAHQqZE
TYP81eFzLQNnPHt4EVVUh7VfDESU84KezmD5QlWpXLmvU31/yMf+Se8xhHTvKSCZIFImWwoG6mb
UoWf9nzpIoaSjB+weqqUumpaaasXVal72J+UX2B+2RPW3RcT0e0zQggqJL3RkrTJvdsjE3JEAvg
q3lGHSZXy28G3skua2SmVi/w4yCE6gb0DqnTWlg7+wC604ydGXA8VJiS5ap43JXiUFFAaQ==
EOF



SA 계정으로 접근(토큰)

바로 secret자원에 접근이 불가능하기 때문에, SA계정을 통해서 접근을 권장한다.
보통 토큰으로 접근 시 이 방법을 많이 사용한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: Tekton-sa  
secrets:  
  - name: git-auth  
EOF
```



토큰 인증 및 실행

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: TaskRun
metadata:
  generateName: git-token-
spec:
  serviceAccountName: kubernetes-github-sa-token
  params:
    - name: private-repo
      value: https://github.com/tangt64/duststack-osp-auto.git
  taskRef:
    name: demo-github-list
EOF
```



SSH 인증 및 실행

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1beta1
kind: TaskRun
metadata:
  generateName: git-ssh-
spec:
  serviceAccountName: kubernetes-github-sa-ssh
  params:
    - name: private-repo
      value: tangt64@github.com:tangt64/dustbox-auto-osp
  taskRef:
    name: demo-github-list
EOF
```



테크톤 기능확장

Tekton HUB는 안녕!

Tekton Hub란?

Tekton Hub는 Tekton Pipeline에서 사용할 Task/Pipeline을 공유하는 공식 Catalog 서비스였다.

주요 기능

1. Tekton Task/Pipeline 검색 및 공유
2. 재사용 가능한 CI/CD 컴포넌트 제공
3. 커뮤니티 기반 CI/CD 자동화 템플릿 저장소
4. CLI 또는 YAML에서 직접 참조 가능



Tekton Hub 중지

Tekton 공식 Public Hub(<https://hub.tekton.dev>) 현재 중지가 되었다.
상태

- Deprecated (사용 중지 예정)
- 2026년 1월 7일 공식 종료

중지 이유는 다음과 같다.

1. 기능 중복

- Helm
- OLM
- OCI artifacts
- Git 기반 catalog



Tekton Hub 중지

2. Cloud Native Artifact 관리 통합

CNCF는 모든 artifact catalog를 하나로 통합

- Helm charts
- OPA policies
- Tekton Tasks
- Kubernetes operators

→ 하나의 플랫폼에서 검색이 가능하도록 통합.



깃 저장소에서 미러링

기존방식에서 깃서버 기반으로 전환을 권장하고 있다. 전환 시 다음과 같은 장점이 있다.

1. 사내 Task catalog 운영 가능
2. 버전 관리(Git)
3. PR/Code Review 가능
4. 폐쇄망 환경 지원
5. CI/CD 자동화 연동



Tekton Hub → Artifact Hub 전환

현재 Tekton Task/Pipeline Catalog는 <https://artifacthub.io>에서 관리.

핵심 정리

Tekton Hub

- Tekton Task/Pipeline catalog 서비스
- 2026년 1월 7일 종료

대체 서비스

- Artifact Hub

현재 표준

- Tekton Tasks / Pipelines

Artifact	예
Helm Chart	nginx
Tekton Task	git-clone
Tekton Pipeline	buildpacks
OPA Policy	security policy
Operators	OLM operator



허브(이전방식)

확장 기능을 통해서 몇가지 기능을 구현한다. 아래와 같이 검색 및 설치를 진행한다.

```
tkn hub get task
tkn hub search
tkn search buildah
tkn search kubernetes
tkn search git-clone
tkn install buildah
tkn install kubernetes-action
tkn install git-clone
```



허브(깃 방식 설치)

앞으로는 깃 혹은 아티팩트 기반으로 설치해야 한다. 여기서는 깃 기반으로 구성한다.

```
git clone https://github.com/tektoncd/catalog  
cd catalog
```

```
kubectl apply -f task/git-clone/0.9/git-clone.yaml  
kubectl apply -f task/buildah/0.6/buildah.yaml  
kubectl apply -f task/kubernetes-actions/0.2/kubernetes-actions.yaml
```



TKN/METALLB 설치 확인

트리거가 설치가 되어 있는지 확인한다. 트리거를 사용하기 위해서 로드밸런서 서비스를 구성 후, 테스트 하도록 한다.

```
tkn version
```

```
Client version: 0.36.0
```

```
Pipeline version: v0.59.0
```

```
Triggers version: v0.26.2
```



METALLB 설치 확인

L4/L7서비스를 간단하게 MetalLB기반으로 구성한다. MetalLB를 사용하기 어려우면 NodePort기반으로 구성하여도 상관 없다.

```
helm repo add metallb https://metallb.github.io/metallb
helm repo update
helm install metallb metallb/metallb --namespace metallb-system \
--create-namespace
```



INGRESS 설치 확인

설치가 안되어 있는 경우, Ingress를 Helm기반으로 설치한다.

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx --namespace ingress-nginx --create-namespace
```

```
kubectl get pods -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS	
AGEingress-nginx-controller	1/1	Running	0	2m



테크톤 트리거 구성 및 설정

webhook

트리거 소개

트리거(trigger)는 웹훅(webhooks)를 통해서 외부에서 테크톤 작업을 실행 시킨다.

예를 들어서 외부/내부에서 사용하는 깃 서버에서 웹훅을 구성하여, 소스코드가 업데이트가 되면 서비스를 업데이트를 위해서 빌드를 자동으로 수행한다.

보통 낮은 사양으로 랩을 진행하기 때문에 여기서는 GITLAB대신 Gogs 및 docker-registry를 사용하여 가볍게 구현 및 시연한다.

1. MetalLB
2. Gitea
3. Docker-registry(v2)
4. GITLAB(사용하시는 워크스테이션 및 노트북에 많은 메모리가 요구가 됩니다.)



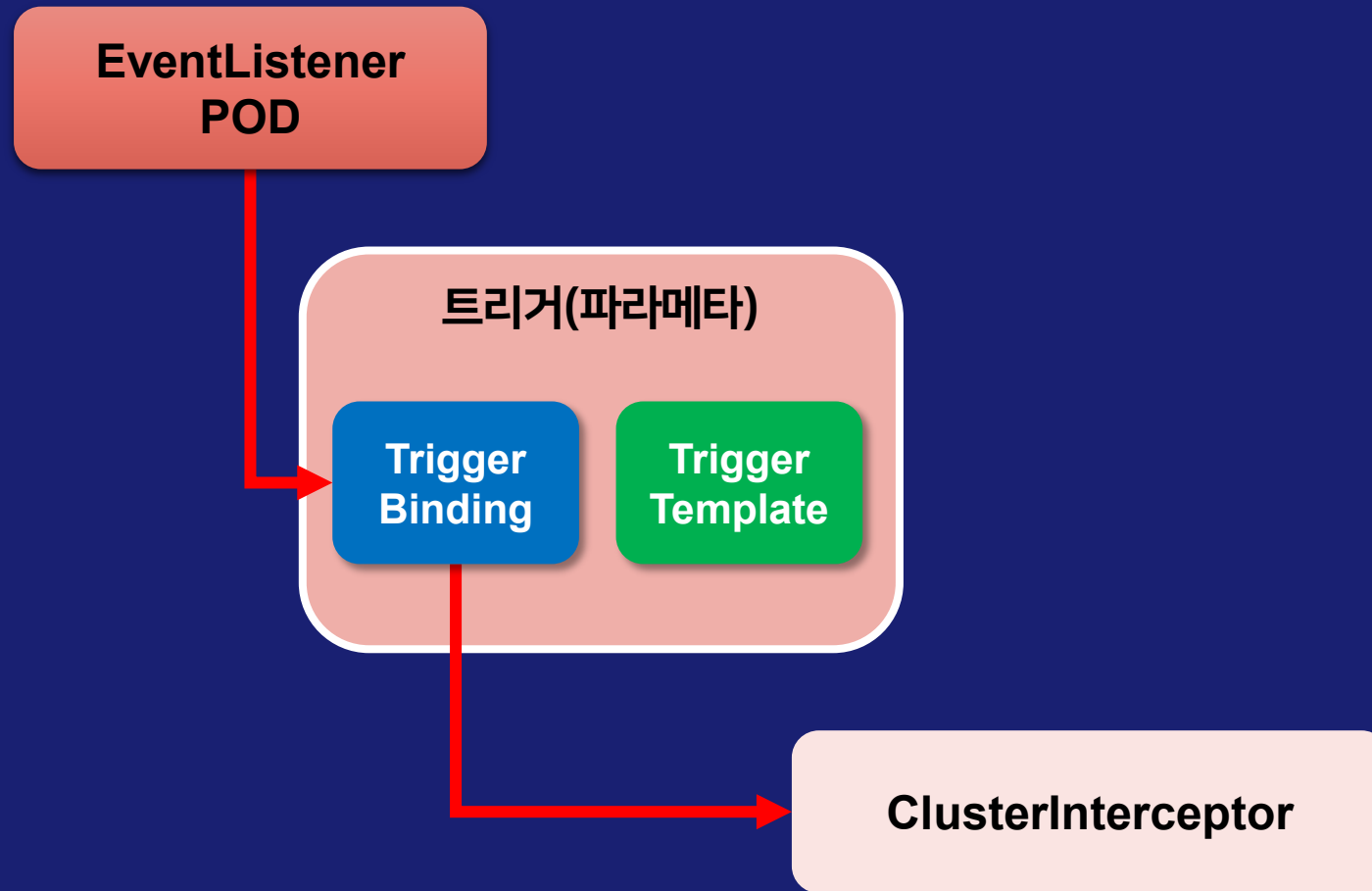
트리거 설명

트리거를 구성 및 실행하기 위해서, 어떠한 자원을 외부에서 웹훅(webhook)를 통해서 실행이 가능한지 선언해야 한다. 이러한 테크톤 자원은 트리거 바인딩(trigger binding)이라고 부른다.

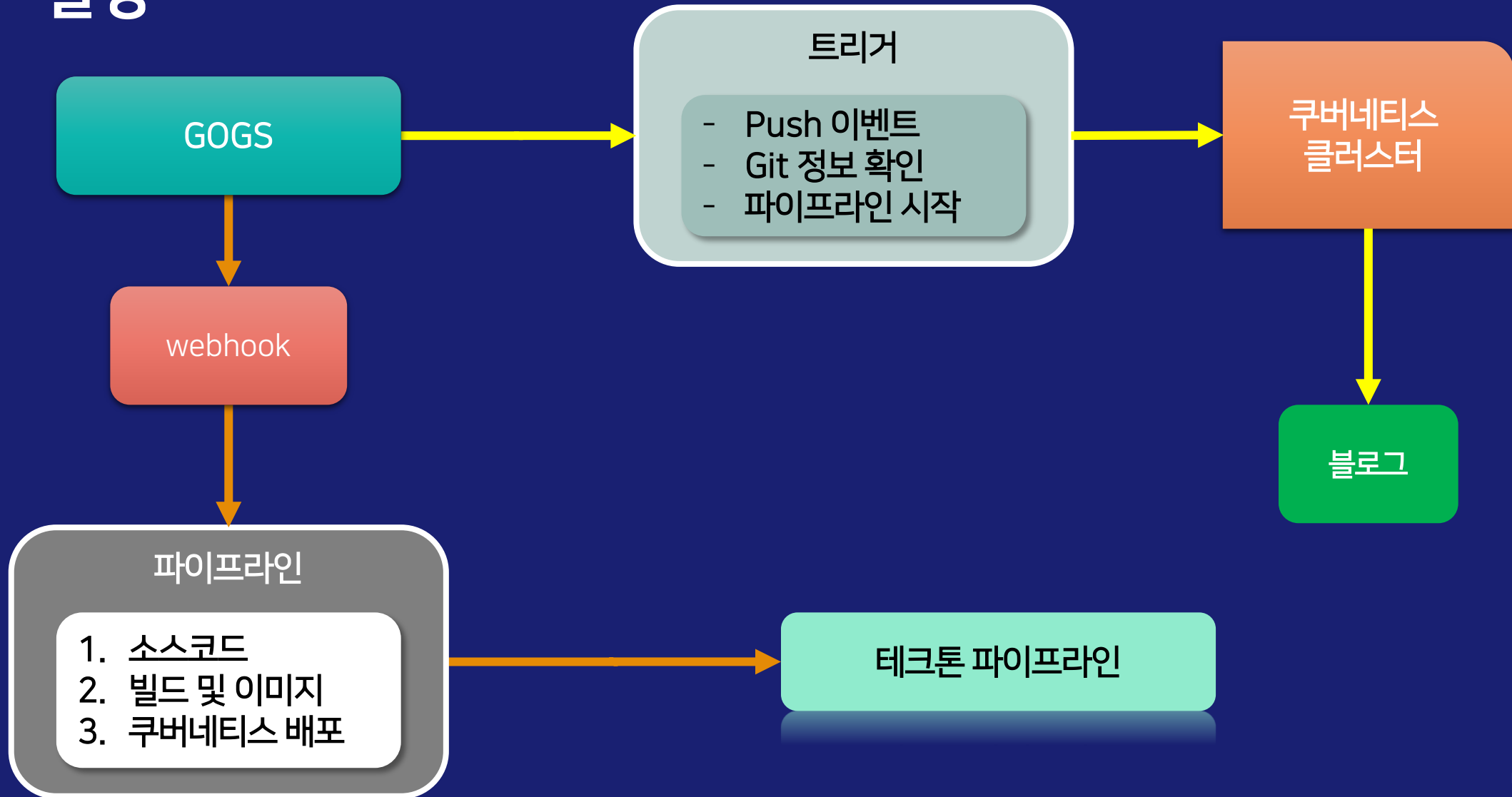
트리거 자원은 별도 자원으로 제공 및 구성이 되기 때문에 따로 설치해야 한다. 이 랩은 앞에서 미리 트리거를 설치하였다.



동작방식



설명



트리거 리소스 소개

트리거 관련 자원을 다음과 같이 테크톤에서 제공하고 있다.

리소스 종류	설명
EventListener	외부 이벤트를 수신하는 진입점. HTTP 엔드포인트 제공.
Trigger	수신된 이벤트를 바탕으로 어떤 작업을 할지 정의.
TriggerBinding	이벤트에서 가져올 파라미터 정의.
Interceptor	이벤트 필터링 또는 인증 처리.
TriggerTemplate	PipelineRun 또는 TaskRun을 생성하는 템플릿.
PipelineRun/TaskRun	실제 실행되는 파이프라인 또는 태스크 인스턴스.



트리거 리소스 필드

각각 트리거 자원은 아래 구성 요소와 관련이 있다. 각 리소스는 아래와 같이 보통 서로 관련이 있다.

리소스 종류	주요 필드/요소
EventListener	serviceName, triggers, bindings, template
Trigger	name, bindings, template, interceptors
TriggerBinding	params.name, params.value
Interceptor	webhook, params, ref
TriggerTemplate	params, resourcetemplates
PipelineRun/TaskRun	pipelineRef, params, workspaces



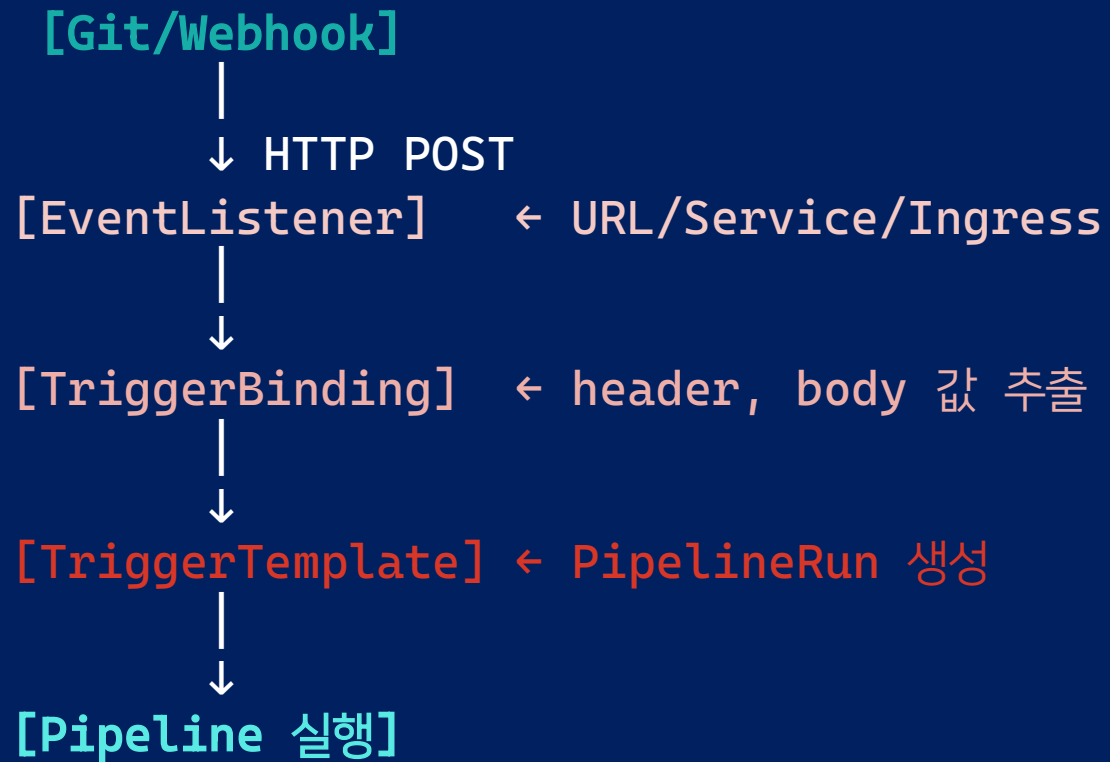
WEBHOOK 조건

웹훅을 사용 시, 아래와 같이 조건을 구성한다. URL이 다른 경우, 다른 URL를 사용하여도 상관 없다.

항목	값
Target URL	http://tekton-webhook.vlab.dustbox.kr
Content Type	application/json (또는 application/x-www-form-urlencoded, Tekton에 따라 다름)
Secret	아무 문자열 (예: tekton-secret) - TriggerBinding에서 사용
Trigger On Events	Push events (또는 원하는 이벤트)



작업 처리 순서



GOGS/Gitea WEBHOOK

Payload URL은 말 그대로 POST 데이터를 전달 받는 위치이다. 사용하는 URL를 적어준다.

Settings

Options

Collaboration

Branches

Webhooks

Git Hooks

Deploy Keys

Add webhook

Gogs will send a POST request to the URL you specify, along with details regarding the event that occurred. You can also specify what kind of data format you'd like to get upon triggering the hook (JSON, x-www-form-urlencoded, XML, etc). More information can be found in our [Webhooks Guide](#).

Payload URL *

This connection is not secure. Logins entered here could be compromised. [Learn More](#)

Manage Passwords

Secret

Secret will be sent as SHA256 HMAC hex digest of payload via X-Gogs-Signature header.

When should this webhook be triggered?

☒ Just the push event

☐ I need **everything**

☐ Let me choose what I need



GOGS/Gitea WEBHOOK

특별한 조건이 없으면 Just the push event으로 구성한다.

Git Hooks

Deploy Keys

Content Type

application/json

Secret

Secret will be sent as SHA256 HMAC hex digest of payload via X-Gogs-Signature header.

When should this webhook be triggered?

☒ Just the push event

☐ I need **everything**

☐ Let me choose what I need

☒ Active

Details regarding the event which triggered the hook will be delivered as well.

Add webhook



이미지 빌드 디스크(PVC)

이미지 빌드를 위한 컨테이너 디스크를 구성한다. 앞에서 사용한 저장소와 동일하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: react-blog-pvc
  namespace: react-blog
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

EOF



WEBHOOK INGRESS 설정 1

사용이 가능하면 인그레스 서비스를 구성한다.

사용이 어려운 경우, 도메인이 필요 없는 경우 nodeport로 구성하여도 상관이 없다. 상황에 따라서 적절하게 사용한다. 여기는 TLS 없이 진행.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: Tekton-listner
  namespace: react-blog
  annotations:
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
EOF
```



WEBHOOK INGRESS TLS 설정 1

TLS키가 있는 경우 아래처럼 추가한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tekton-webhook-ingress
  namespace: react-blog
  annotations:
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
  tls:
    - hosts:
      - tekton-webhook.vlab.dustbox.kr
      secretName: tekton-webhook-tls # 미리 만들어둔 TLS Secret
```

계
속



WEBHOOK INGRESS

앞에 내용 계속 이어서... 반드시, A 레코드를 dnsmasq나 혹은 bind에서 구성해야 한다.

```
rules:
  - host: tekton.example.com
    http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: el-react-blog
              port:
                number: 8080
```



EOF



트리거 서비스 계정

트리거를 올바르게 동작하기 위해서 트리거 서비스 계정을 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: tekton-triggers-sa
  namespace: react-blog
EOF
```



트리거 SA 계정 역할 구성 1

트리거 SA 계정이 사용할 ROLE/ROLEBINDING를 구성한다. 이 정보는 테크톤 매뉴얼에 있는 내용이다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: Tekton-triggers-role
  namespace: react-blog
rules:
- apiGroups: ["triggers.tekton.dev"]
  resources: ["eventlisteners", "triggerbindings", "triggertemplates",
"triggers"]
  verbs: ["get", "list", "watch"]
```



계
속

트리거 SA 계정 역할 구성 2

```
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["tekton.dev"]
  resources: ["pipelineruns", "pipelineresources", "taskruns"]
  verbs: ["create"]
- apiGroups: [""]
  resources: ["serviceaccounts"]
  verbs: ["impersonate"]
```

EOF



트리거 SA 계정 역할 바인딩 1

네임스페이스 ROLE BINDING를 구성 및 생성한다. 네임 스페이스 선언 필요 없다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: rbac.authorization.k8s.io/v1  
kind: RoleBinding  
metadata:  
  name: tekton-triggers-roleBinding  
  namespace: react-blog  
subjects:  
- kind: ServiceAccount  
  name: tekton-triggers-sa
```



트리거 SA 계정 역할 바인딩 2

```
roleRef:  
  apiGroup: rbac.authorization.k8s.io  
  kind: Role  
  name: tekton-triggers-role  
EOF
```



트리거 SA 계정 클러스터 권한 1

만약, 다른 네임스페이스 자원도 동시에 관리하는 경우, 클러스터 역할로 구성한다.

```
cat <<'EOF' | kubectl apply -f -
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: tekton-triggers-clusterrole
rules:
- apiGroups: ["triggers.tekton.dev"]
  resources: ["clustertriggerbindings", "clusterinterceptors"]
  verbs: ["get", "list", "watch"]
EOF
```



트리거 SA 계정 클러스터 권한 2

서비스 계정이 사용할 클러스터 역할을 할당.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: tekton-triggers-clusterbinding
  namespace: react-blog
subjects:
- kind: ServiceAccount
  name: tekton-triggers-sa
  namespace: react-blog
```



트리거 SA 계정(클러스터 역할) 2

위의 내용 계속 이어서...

```
roleRef:  
  apiGroup: rbac.authorization.k8s.io  
  kind: ClusterRole  
  name: tekton-triggers-clusterrole  
EOF
```



트리거 바인딩 처리 1

HTTP 요청의 Header 및 Payload에서 Pipeline 실행에 필요한 값을 추출하여 바인딩 처리.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: git-binding
  namespace: react-blog
spec:
  params:
    - name: gitrepositoryurl
      value: $(body.repository.clone_url)
    - name: gitrevision
      value: $(body.after)
```

EOF



트리거 템플릿 설명

TriggerBinding으로 전달된 값을 사용해 PipelineRun(또는 TaskRun)을 생성하는 템플릿.

- 실제로 무엇을 실행할지를 정의
- Pipeline 실행의 출발점을 찍는 역할
- 변수 치환을 통해 동적 실행 가능

기능	설명
PipelineRun 생성	어떤 Pipeline을 실행할지 정의
파라미터 치환	Binding에서 전달된 값 적용
ServiceAccount 지정	실행 권한 결정
Workspace 정의	PVC / EmptyDir 연결



트리거 템플릿 구성 1

```
cat <<'EOF' | kubectl apply -f -
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
metadata:
  name: tt-react-blog
spec:
  params:
    - name: gitrevision
    - name: gitrepositoryurl
```



트리거 템플릿 구성 2

```
resourcetemplates:  
  - apiVersion: tekton.dev/v1  
    kind: PipelineRun  
    metadata:  
      generateName: react-blog-run-  
spec:  
  serviceAccountName: sa-blog  
  pipelineRef:  
    name: react-blog-pipeline
```



트리거 템플릿 구성 3

트리거 템플릿 구성.

params:

- name: repo-url
value: \$(params.gitrepositoryurl)
- name: repo-revision
value: \$(params.gitrevision)
- name: image-url
value: registry.example.com/react-blog/



트리거 템플릿 구성 4

```
workspaces:  
  - name: shared-data  
    volumeClaimTemplate:  
      metadata:  
        name: react-blog-pvc  
      spec:  
        accessModes: ["ReadWriteOnce"]  
        resources:  
          requests:  
            storage: 1Gi
```



트리거 템플릿 구성 5

```
workspaces:  
  - name: shared-data  
    persistentVolumeClaim:  
      claimName: react-blog-pvc
```

EOF



최종 작업 처리 파이프 라인 구성 1

```
cat <<'EOF' | kubectl apply -f -
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: react-blog-pipeline
  namespace: react-blog
spec:
  params:
    - name: repo-url
    - name: repo-revision
    - name: image-url
  workspaces:
    - name: shared-data
```



최종 작업 처리 파이프 라인 구성 2

```
tasks:
- name: clone-source
  taskRef:
    name: git-clone
    kind: ClusterTask
  params:
    - name: url
      value: $(params.repo-url)
    - name: revision
      value: $(params.repo-revision)
    - name: deleteExisting
      value: "true"
  workspaces:
    - name: output
      workspace: shared-data
```

계
속



최종 작업 처리 파이프 라인 구성 4

```
- name: build-image
  runAfter: [clone-source]
  taskRef:
    name: buildah
    kind: ClusterTask
  params:
    - name: IMAGE
      value: $(params.image-url)
    - name: CONTEXT
      value: $(workspaces.source.path)
    - name: TLSVERIFY
      value: "false"      # insecure registry 설정
```

계
속



최종 작업 처리 파이프 라인 구성 5

workspaces:

- name: source

workspace: shared-data



최종 작업 처리 파이프 라인 구성 6

```
- name: deploy-app
  taskRef:
    name: kubernetes-action
  params:
    - name: script
      value: |
        kubectl apply -f deploy.yaml
```

EOF



이벤트 리스너 설명

외부 시스템(Git, Webhook 등)에서 들어오는 HTTP 이벤트를 수신하는 Tekton Triggers의 진입점.

- HTTP Endpoint 제공
- 이벤트 수신 시 Trigger 실행
- Binding+Template을 연결하는 허브



이벤트 리스너 구성 1

이벤트 리스너.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: el-goblog-listener
  namespace: react-blog
spec:
```



이벤트 리스너 구성 2

이벤트 리스너.

```
serviceAccountName: sa-blog

triggers:
- name: react-blog-trigger
  bindings:
    - ref: git-binding
  template:
    ref: tt-react-blog
```

EOF



ArgoCD 설치 및 구성

설치 및 테스트

ArgoCD 간단한 설명

Argo CD는 Git 저장소를 Single Source of Truth로 사용하여 Kubernetes 애플리케이션을 자동으로 배포하고 동기화하는 GitOps 기반 Continuous Delivery 도구이다.

애플리케이션의 원하는 상태(Desired State)를 Git에 정의하고, 클러스터 상태와 지속적으로 비교하여 자동으로 동기화한다.

상세 과정은 CI/CD 개발자 과정 및 ArgoCD 과정에서 다룬다. 여기서는 단일 네임스페이스 기반으로 진행한다.



ArgoCD 핵심 기능

GitOps 기반 배포

- Git repository가 인프라 및 애플리케이션의 기준 상태

자동 동기화 (Auto Sync)

- Git 변경 → Kubernetes 자동 반영

Drift Detection

- 클러스터 설정 변경 감지 후 자동 복구

멀티 클러스터 관리

- 여러 Kubernetes 클러스터 중앙 관리 가능

웹 UI 제공

- 애플리케이션 상태 시각적으로 확인



ArgoCD 설치

설치는 아래 명령어로 진행한다. 현재 공식 설치하는 YAML Manifest만 지원하고 있다.

```
kubectl create namespace argocd
kubectl apply --server-side -n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
```

```
kubectl get pods -n argocd
argocd-server
argocd-repo-server
argocd-application-controller
argocd-dex-server
argocd-redis
```



ArgoCD 대시보드

대시보드 접근을 위해서 기존 ClusterIP를 NodePort로 변경한다.

```
kubectl patch svc argocd-server -n argocd \
-p '{"spec": {"type": "NodePort"}}'
kubectl get svc argocd-server -n argocd
argocd-server NodePort 443:32080/TCP
```



외부 네트워크 접근 도메인 구성(선택사항)

외부에서 접근 Ingress 혹은 HAProxy로 아래와 같이 설정한다.

```
kubectl apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: argocd
  namespace: argocd
spec:
  ingressClassName: nginx
```



외부 네트워크 접근 도메인 구성

```
rules:
  - host: argocd.dustbox.kr
    http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: argocd-server
              port:
                number: 80
```

EOF



INSECURE REGISTRY

만약, TLS가 SERVER CA혹은 공식 ROOT CA를 사용하지 않는 경우, 다음과 같이 꼭 설정한다.

```
cat >/etc/containers/registries.conf.d/registry-dustbox.conf <<EOF
[[registry]]
location = "registry.dustbox.kr"
insecure = true
EOF
```



ArgoCD 비밀번호 확인

대시보드 접근을 위한 ArgoCD 비밀번호를 확인.

```
kubectl get secret argocd-initial-admin-secret \
-n argocd \
-o jsonpath="{.data.password}" | base64 -d
user: admin
password: 출력된 값
```



ArgoCD CLI

CLI도구가 필요한 경우, 아래와 같이 설치한다.

```
curl -sSL -o argocd \  
https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-  
amd64  
chmod +x argocd  
sudo mv argocd /usr/local/bin/  
  
argocd login argocd.dustbox.kr --insecure
```



GIT 서버 접근 설정

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: repo-lab-react-blog
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
EOF
```



GIT 서버 접근 설정

```
stringData:  
  url: https://git.dustbox.kr/tang/react-blog.git  
#  username: tang  
#  password: GIT_PASSWORD  
  insecure: "true"  
  type: git  
EOF
```

ID/PW가 필요 없는 경우 무시



간단한 애플리케이션 배포

```
kubectl apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: react-blog
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://git.dustbox.kr/tang/lab-react-blog.git
    targetRevision: main
    path: base
EOF
```



간단한 애플리케이션 배포

```
destination:  
  server: https://kubernetes.default.svc  
  namespace: blog
```

```
syncPolicy:  
  automated:  
    prune: true  
    selfHeal: true
```

```
syncOptions:  
  - CreateNamespace=true
```

EOF



ArgoCD 재배포

올바르게 동작이 안되면 아래 명령어로 다시 서버를 재-배포 한다.

```
kubectl rollout restart deployment argocd-repo-server -n argocd  
kubectl rollout restart deployment argocd-server -n argocd
```

